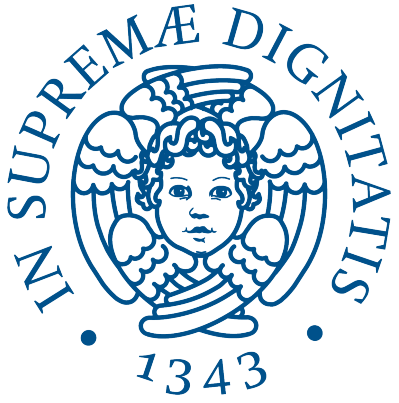

University of Pisa
Master's Degree in Computer Science,
Artificial Intelligence

Computational Mathematics for Learning & Data Analysis (646AA)

Group number: 63
Project number: 25 ML

Members:
Matthew Bernardi
Gabriele Marsili

Academic Year: 2025/2026

Project statement.

25. (P) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem $w_{k+1} = \arg\min_w f_k(w)$, $f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$ where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$.

(A2) is an algorithm of the class of the *deflected subgradient methods*.

Contents

Chapter 1

Introduction to the problem

1.1 Extreme Learning Machine

Extreme Learning Machine (ELM) [15] is a single-hidden-layer neural network, in which:

- $\mathbf{W_1} \in \mathbb{R}^{H \times N}$ is the input-to-hidden weight matrix (where H is the number of nodes in the hidden layer and N is the number of input nodes) that is **randomly generated** and **static** (does not change during training) [15].
- $\sigma(\cdot)$ is the element-wise (nonlinear) activation function applied to $\mathbf{W_1 x} \in \mathbb{R}^H$, where $\mathbf{x} \in \mathbb{R}^N$ is the input vector. We assume from now on that the hidden-layer feature matrix $\mathbf{X} \in \mathbb{R}^{M \times H}$ has full column rank, which is the regime studied in the original ELM analysis [15, Theorem 2.1] for $M \geq H$ with random $\mathbf{W}_1$ and an infinitely differentiable activation (our sigmoid satisfies this). In our experiments ($\mathbf{W}_1$ Gaussian, $M \geq H$) it is verified numerically by the conditioning checks of Section 5.1.1.
- $\mathbf{w} \in \mathbb{R}^H$ is the hidden-to-output weight vector.
- $\hat{y} = \mathbf{w}^T \sigma(\mathbf{W_1 x}) \in \mathbb{R}$ is the prediction of the model, and $y \in \mathbb{R}$ is the ground truth (target).

Once $\mathbf{W_1}$ is fixed, knowing that $\mathbf{X}_{\text{origin}} \in \mathbb{R}^{M \times N}$ is the matrix of all the inputs, the hidden-layer activations $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1^T)$ are constant, and finding $\mathbf{w}$ reduces to solving the Least-Squares problem described in the following section.

1.2 Least-Squares Problem

The definition of the Least Squares Problem [6] in the ELM case is the following:

$$\min_{\mathbf{w}} \frac{1}{2} ||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 \tag{1.1}$$

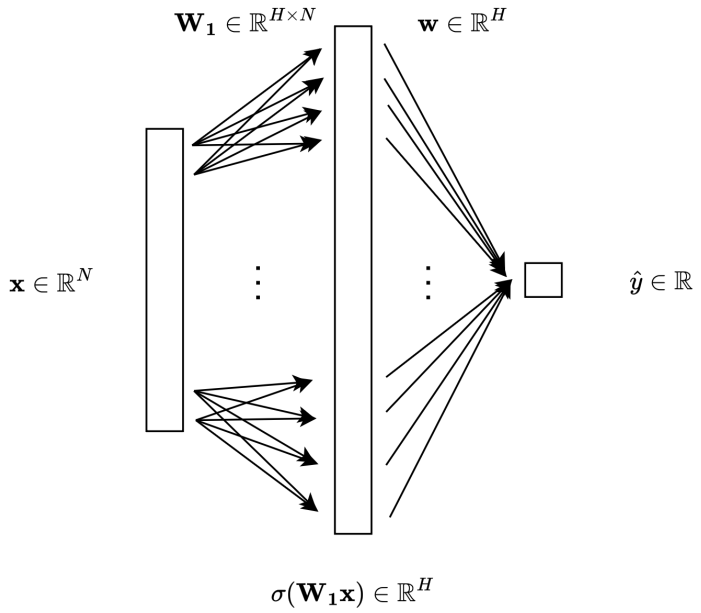

Figure 1.1: Visual representation of an Extreme Learning Machine model.

The Least-Squares problem admits a unique solution if and only if the matrix $\mathbf{X}$ has full column rank; otherwise solutions exist but are not unique. When the rank condition holds, we can rewrite the problem as

$$\min_{\mathbf{w}} \frac{1}{2}\mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{y}^T\mathbf{X}\mathbf{w} + \frac{1}{2}\mathbf{y}^T\mathbf{y} \tag{1.2}$$

whose gradient $\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y} = 0$ uniquely when

$$\mathbf{w}_0 = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}.$$

Adding the L_1 regularization term (Section 1.3) destroys the closed-form structure of the normal equations and requires iterative methods.

1.3 Regularization Techniques

1.3.1 L_1 regularization

When adding the L_1 regularization term to the Least-Squares, the equation is the following:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{2}||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 + \lambda_{\text{LASSO}} \sum_{i=1}^{H} |w_i| \tag{1.3}$$

This form induces $w_i \to 0$ at the cost of non-differentiability where $w_i = 0$, which prevents direct use of gradient methods. We handle the non-smoothness with IRLS (Chapter 2) and the deflected subgradient method (Chapter 3).

1.4 Objective Function Analysis

We decompose the objective as

$$\min_{\mathbf{w}} f(\mathbf{w}) = \underbrace{\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}}\|\mathbf{w}\|_1}_{h(\mathbf{w})} \tag{1.4}$$

and analyse its properties below.

1.4.1 Convexity

The properties of f depend on $\mathbf{X}$.

Proposition 1.1. *If $\mathbf{X} \in \mathbb{R}^{M\times H}$ has full column rank, then $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{LASSO}\|\mathbf{w}\|_1$ is strictly convex and admits a unique global minimizer $\mathbf{w}^*$ [2, p. 73].*

Proof. **Strict convexity.** Since $\mathbf{X}$ has full column rank, for any $\mathbf{v} \neq \mathbf{0}$ we have $\mathbf{X}\mathbf{v} \neq \mathbf{0}$, so $\mathbf{v}^T(\mathbf{X}^T\mathbf{X})\mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 > 0$. Hence the Hessian of g is $\nabla^2 g = \mathbf{X}^T\mathbf{X} \succ 0$, making g strictly convex. h is convex as a non-negative multiple of a norm. The sum of a strictly convex function and a convex function is strictly convex [12, Slides 21–24], so f is strictly convex.

Existence and uniqueness of the minimizer. Since $\mathbf{X}^T\mathbf{X} \succ 0$, g is coercive: $g(\mathbf{w}) \to +\infty$ when $\|\mathbf{w}\| \to \infty$. Because $h \geq 0$, $f = g + h$ is also coercive. This means the sub-level set $\{\mathbf{w} : f(\mathbf{w}) \leq f(\mathbf{0})\}$ is compact, and, since f is continuous, it has its minimum on this set. Strict convexity ensures that there is a single minimizer: if $\mathbf{w}_1 \neq \mathbf{w}_2$ both minimized f, the midpoint $\frac{\mathbf{w}_1+\mathbf{w}_2}{2}$ would give a strictly smaller value, a contradiction. □

1.4.2 Non-smoothness

Unlike the quadratic term, the L_1 penalty is **not differentiable** [10, Slide 4] at any point where some component $w_i = 0$, so f is non-smooth. At points where all $w_i \neq 0$, the gradient exists and is:

$$\nabla f(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}}\,\text{sign}(\mathbf{w})$$

where $\text{sign}(\mathbf{w})$ is applied element-wise. At points where some $w_i = 0$, one must reason in terms of *subgradients* (see Chapter 3).

1.4.3 Optimality conditions

The optimality condition is expressed using the **subdifferential** [10, Slide 5]: $\mathbf{w}^*$ is a global minimizer for the problem if and only if:

$$0 \in \partial f(\mathbf{w}^*) = \mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}}\, \partial \|\mathbf{w}^*\|_1$$

where the subdifferential of the L_1 norm is given component-wise:

$$[\partial \|\mathbf{w}\|_1]_i = \begin{cases} \{+1\} & \text{if } w_i > 0, \\ \{-1\} & \text{if } w_i < 0, \\ [-1, +1] & \text{if } w_i = 0. \end{cases}$$

Writing it in a component-wise form: for each $i = 1, \ldots, H$

$$\left[\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y})\right]_i + \lambda_{\text{LASSO}} s_i = 0 \qquad s_i \in \partial |w_i^*| \tag{1.5}$$

Chapter 2

Algorithm 1: Iteratively Reweighted Least Squares

Iteratively Reweighted Least Squares (IRLS) [8] is a classical algorithm for solving optimization problems involving non-smooth objective functions of the form of a p-norm. It handles non-smoothness by solving a sequence of *weighted* least-squares problems, each of which is smooth and admits a closed-form solution.

2.1 A strict upper bound for the L_1 norm

We solve the differentiability problem replacing the L_1 term with a smooth surrogate function, using the inequality $(|w_i| - |(w_k)_i|)^2 \geq 0$, which, after expansion and division by $2|(w_k)_i|$, gives the majorization:

$$|w_i| \leq \frac{w_i^2}{2|(w_k)_i|} + \frac{|(w_k)_i|}{2} \qquad (w_k)_i \neq 0 \tag{2.1}$$

Summing over $i = 1, \ldots, H$ returns the bound on the L_1 norm

$$\|\mathbf{w}\|_1 \leq \frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w} + \frac{1}{2}\|\mathbf{w}_k\|_1 \tag{2.2}$$

where $\mathbf{W}_k \in \mathbb{R}^{H \times H}$ is a diagonal matrix with entries:

$$(\mathbf{W}_k)_{ii} = |(w_k)_i|^{-1/2} \tag{2.3}$$

This bounds the non-smooth L_1 term using a *smooth, quadratic* expression $\frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$, plus a constant term that depends only on the current iteration $\mathbf{w}_k$. $\mathbf{W}_k^T\mathbf{W}_k$ assigns each component of $\mathbf{w}$ a per-iteration weight $|(w_k)_i|^{-1}$ [5]: components with $|(w_k)_i| \ll 1$ receive a larger penalty (suppressing them), components with $|(w_k)_i| \gg 1$ receive a smaller one.

Handling near-zero components. Equation (2.3) becomes numerically unstable when $(w_k)_i \approx 0$, since $|(w_k)_i|^{-1/2}$ can blow up. To prevent division by (near) zero, a **safety threshold** $\varepsilon_{\text{thr}} > 0$ is introduced:

$$(\mathbf{W}_k)_{ii} = \Big(\max\big(|(w_k)_i|,\, \varepsilon_{\text{thr}}\big)\Big)^{-1/2} \tag{2.4}$$

Typical values are $\varepsilon_{\text{thr}} \in [10^{-6}, 10^{-10}]$.

2.2 The Majorization-Minimization interpretation

IRLS fits inside the Majorization-Minimization (MM) framework [16]. Instead of minimizing f directly, at each iteration k we build a *surrogate* $Q(\mathbf{w}, \mathbf{w}_k)$ function that:

1. Majorizes f: $Q(\mathbf{w}, \mathbf{w}_k) \geq f(\mathbf{w})$ for all $\mathbf{w}$
2. Touches f at $\mathbf{w}_k$: $Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$
3. Is smooth, so that it can be minimized in closed form

Plugging the quadratic upper bound (2.2) into (1.4) gives the following surrogate:

$$Q(\mathbf{w}, \mathbf{w}_k) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 \tag{2.5}$$

We need to check that the properties are satisfied:

1. Majorization property: follows from (2.1)
2. Touching property: at $\mathbf{w} = \mathbf{w}_k$, we get the following Q function

   $$Q(\mathbf{w}_k, \mathbf{w}_k) = \tfrac{1}{2}\|\mathbf{X}\mathbf{w}_k - \mathbf{y}\|_2^2 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 = f(\mathbf{w}_k)$$

3. **Smoothness property:** The surrogate $Q(\mathbf{w}, \mathbf{w}_k)$ is a function of $\mathbf{w}$, since $\mathbf{w}_k$ is fixed (constant). Its gradient is

   $$\nabla_{\mathbf{w}} Q(\mathbf{w}, \mathbf{w}_k) = \left(\mathbf{X}^T\mathbf{X} + \lambda_{\text{LASSO}}\mathbf{W}_k^T\mathbf{W}_k\right)\mathbf{w} - \mathbf{X}^T\mathbf{y}$$

   Since it is an affine function of $\mathbf{w}$, the Hessian is a constant matrix. Because of that, Q is infinitely differentiable (C^∞), and also globally L-smooth (with a Lipschitz continuous gradient).

Taking $\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} Q(\mathbf{w}, \mathbf{w}_k)$, the first two MM properties combined give $f(\mathbf{w}_{k+1}) \leq Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$, so the sequence $\{f(\mathbf{w}_k)\}$ is non-increasing.

2.3 The weighted least-squares subproblem

To find $\mathbf{w}_{k+1}$, we must minimize the surrogate function $Q(\mathbf{w}, \mathbf{w}_k)$ with respect to $\mathbf{w}$:

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 \right)$$

Since the term $\frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1$ is constant with respect to $\mathbf{w}$, it does not affect the optimization and can be dropped. The minimization problem simplifies to a standard **weighted least-squares problem with L_2 regularization**:

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{IRLS}}\|\mathbf{W}_k\mathbf{w}\|_2^2 \right) \tag{2.6}$$

using:

$$\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}} \tag{2.7}$$

Proposition 2.1 (Weighted normal equation solution)**.** *The unique solution of* (2.6) *satisfies:*

$$\left(\mathbf{X}^T\mathbf{X} + 2\lambda_{IRLS}\mathbf{W}_k^T\mathbf{W}_k\right)\mathbf{w}_{k+1} = \mathbf{X}^T\mathbf{y} \tag{2.8}$$

Proof. The gradient of the objective function (2.6) is:

$$\nabla f_k(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$$

Setting this to zero results in (2.8). The coefficient matrix $\mathbf{X}^T\mathbf{X}+2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is symmetric positive definite (it is the sum of $\mathbf{X}^T\mathbf{X} \succeq 0$ and $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k \succ 0$ since $\varepsilon_{\text{thr}} > 0$), so the system has a unique solution. □

Since $\mathbf{W}_k$ is diagonal, defining $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$, equation (2.8) reduces to:

$$\mathbf{Q}_k\,\mathbf{w}_{k+1} = \mathbf{b} \tag{2.9}$$

This is an $H \times H$ symmetric positive definite linear system, which can be solved efficiently at each iteration. Note that $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$ can be precomputed once before the loop, so only the diagonal $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is updated at each iteration.

Solving the linear system. Because $\mathbf{Q}_k$ is SPD, two efficient methods can be used:

- **Cholesky factorization:** [18] computes $\mathbf{Q}_k = \mathbf{L}\mathbf{L}^T$ in $O(H^3/3)$ operations, then solves two triangular systems in $O(H^2)$. This is the direct method of choice for moderate-sized problems.
- **Conjugate Gradient (CG):** [18] an iterative method that converges in at most H steps in exact arithmetic, with convergence rate depending on the condition number $\kappa(\mathbf{Q}_k)$. Preferable for large-scale problems where H is too large for Cholesky.

2.4 The complete algorithm

Algorithm 1 Iteratively Reweighted Least Squares (IRLS)

Require: $\mathbf{X} \in \mathbb{R}^{M \times H}$, $\mathbf{y} \in \mathbb{R}^M$, $\lambda_{\text{LASSO}} > 0$, $\varepsilon_{\text{thr}} > 0$, $\varepsilon_{\text{stop}} > 0$, $k_{\max}$

Pre-compute $\mathbf{A} \leftarrow \mathbf{X}^T\mathbf{X}$, $\mathbf{b} \leftarrow \mathbf{X}^T\mathbf{y}$
Define $\lambda_{\text{IRLS}} \leftarrow \frac{1}{2}\lambda_{\text{LASSO}}$
Initialize $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1}\mathbf{b}$ (ordinary least-squares solution)
for $k = 0, 1, 2, \ldots, k_{\max}$ **do**
    Compute weights: $(\mathbf{W}_k)_{ii} \leftarrow \big(\max(|(w_k)_i|, \varepsilon_{\text{thr}})\big)^{-1/2}$ for all i
    Assemble system: $\mathbf{Q}_k \leftarrow \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$
    Solve: $\mathbf{Q}_k\,\mathbf{w}_{k+1} = \mathbf{b}$ *(via Cholesky or CG)*
    if $\|\mathbf{w}_{k+1} - \mathbf{w}_k\| \,/\, \max(1, \|\mathbf{w}_k\|) < \varepsilon_{\text{stop}}$ **then**
        break *(convergence reached)*
    end if
end for
return $\mathbf{w}_{k+1}$

Initialization. We initialize $\mathbf{w}_0$ with the unregularized least-squares solution $\mathbf{A}^{-1}\mathbf{b}$ (**warm start**): the weights $(\mathbf{W}_0)_{ii} = |(w_0)_i|^{-1/2}$ are well defined without having to lean on ε_{thr} at the first step. A **cold start** $\mathbf{w}_0 = \mathbf{0}$ places all weights at ε_{thr}, delaying the first iterations. In the implementation we add a vanishing ridge $10^{-12}\mathbf{I}$ to $\mathbf{A}$ before this first solve: numerically indistinguishable from $\mathbf{A}^{-1}\mathbf{b}$ when $\mathbf{A} \succ 0$, it keeps the warm start well-posed if $\mathbf{X}$ is rank-deficient.

Stopping criterion. The algorithm ends when the relative change between successive iterations goes below $\varepsilon_{\text{stop}}$:

$$\frac{\|\mathbf{w}_{k+1} - \mathbf{w}_k\|}{\max(1, \|\mathbf{w}_k\|)} < \varepsilon_{\text{stop}}$$

where the $\max(1, \|\mathbf{w}_k\|)$ normalization makes the test scale-invariant and prevents false triggers when $\|\mathbf{w}_k\|$ is small.

2.5 Convergence analysis

Monotone decrease. IRLS monotonically decreases the function. Since f is bounded below by zero, the sequence $\{f(\mathbf{w}_k)\}$ converges.

Theorem 2.2 (Fixed-point consistency)**.** *If the sequence $\{\mathbf{w}_k\}$ generated by IRLS converges to a point $\mathbf{w}^*$ such that $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i, then $\mathbf{w}^*$ satisfies* (1.5)*.*

Proof. At a fixed point $\mathbf{w}_k = \mathbf{w}_{k+1} = \mathbf{w}^*$, (2.8) gives:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + 2\lambda_{\text{IRLS}}\,\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^* = \mathbf{0}$$

Since $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i, the threshold is inactive and $(\mathbf{W}_*^T\mathbf{W}_*)_{ii} = |(w^*)_i|^{-1}$. The i-th component of the regularizer term is:

$$(\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^*)_i = \frac{(w^*)_i}{|(w^*)_i|} = \operatorname{sign}((w^*)_i)$$

Using (2.7):

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}} \operatorname{sign}(\mathbf{w}^*) = \mathbf{0}$$

Since $|(w^*)_i| \neq 0$ for all i, the L_1 norm is differentiable at $\mathbf{w}^*$, so $\partial\|\mathbf{w}^*\|_1 = \operatorname{sign}(\mathbf{w}^*)$ [10]. By the sum rule for subdifferentials [12], the equation above is exactly $\mathbf{0} \in \partial f(\mathbf{w}^*)$, the optimality condition in (1.5) is met. □

Convergence rate. We expect a linear rate empirically: $\|\mathbf{w}_{k+1} - \mathbf{w}^*\|$ contracts by a factor governed by the conditioning of $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$, whose second term regularises the small components and improves the conditioning of $\mathbf{X}^T\mathbf{X}$ via a diagonal shift. A formal linear-rate result for IRLS on ℓ_q minimisation ($q \leq 1$) appears in [8, Theorem 5.3] under a *null space property* (NSP) on the matrix $\mathbf{X}$ (no sparse vector is in the kernel of $\mathbf{X}$). That hypothesis does not hold in general for ELM features and we are not aware of a sparsity-recovery guarantee for the random $\mathbf{W}_1$ regime; we use Daubechies et al. only as the closest available analysis. On the synthetic instance IRLS recovers the planted support exactly (Section 5.5.1); the real datasets have no ground-truth support (Section 5.1.1). The linear rate is verified empirically in Section 5.2.

Verification of hypotheses for the ELM problem. The two assumptions of Theorem 2.2 are satisfied for (1.4) as follows.

1. *Convergence of the sequence.* Under the assumption that $\mathbf{X}$ has full column rank, f is strictly convex and admits a unique minimizer $\mathbf{w}^*$ (Proposition 1.1). IRLS monotonically decreases f (Section 2.2) and the level sets of f are compact by coercivity, so $\{\mathbf{w}_k\}$ has at least one accumulation point. The MM descent identity forces any accumulation point $\bar{\mathbf{w}}$ to be a fixed point of the IRLS map. By Theorem 2.2 every fixed point with $|(\bar{w})_i| > \varepsilon_{\text{thr}}$ for all i satisfies the optimality condition, and by uniqueness $\bar{\mathbf{w}} = \mathbf{w}^*$, so the whole sequence converges.

2. *Non-zero components at convergence.* The hypothesis $|(w^*)_i| > \varepsilon_{\text{thr}}$ fails exactly on the inactive set $I = \{i : (w^*)_i = 0\}$ of the sparse LASSO solution. Strictly, IRLS therefore does not solve the original problem (1.4) but the smoothed variant induced by the clip (2.4) on the weights, in which the L_1 term is replaced by a surrogate that agrees with $|w_i|$ for $|w_i| > \varepsilon_{\text{thr}}$ and with a quadratic otherwise. The minimiser of this smoothed problem differs from the LASSO minimiser by at most $O(\varepsilon_{\text{thr}})$ in objective and recovers the LASSO solution as $\varepsilon_{\text{thr}} \to 0$. On the active set $A = \{i : |(w^*)_i| > \varepsilon_{\text{thr}}\}$ Theorem 2.2 applies and reproduces the LASSO optimality condition exactly. On I the smoothed gradient at $(\bar{w})_i \leq \varepsilon_{\text{thr}}$

pins the component near zero and the corresponding row of (1.5) is satisfied by some $s_i \in [-1, 1]$, in agreement with the subdifferential of $|w_i|$ at 0. The bound $\varepsilon_{\text{thr}} = 10^{-8}$ that we adopt (Section 5.4) is tight enough that this approximation is not detectable against the Clarabel reference (Section 5.6).

2.6 Computational cost

The computational cost of each iteration of IRLS is the following:

- **Weight update $\mathbf{W}_k$**: $O(H)$ simply iterating over the components of $\mathbf{w}_k$.
- **Matrix update $\mathbf{Q}_k = \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$**: $O(H)$, since $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is pre-computed and only the diagonal changes.
- **Linear system solve $\mathbf{Q}_k\mathbf{w}_{k+1} = \mathbf{b}$**: $O(H^3/3)$ with Cholesky, or $O(pH^2)$ with CG for p CG steps (in the following we assume using Cholesky).

Also, we need $O(MH^2)$ for the $\mathbf{A}$ matrix computation and $O(MH)$ for the $\mathbf{b}$ vector creation, and, in case of warm start, we need the OLS solution $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1}\mathbf{b}$, computed in $O(H^3)$. The dominant per-iteration cost is the linear system solve: $O(H^3)$ with Cholesky. Since IRLS typically converges in 50–100 iterations, the $O(MH^2)$ pre-computation is paid back over the iteration loop. The overall cost is therefore:

$$\underbrace{O(MH^2) + O(MH) + O(H^3)}_{\text{pre-computation}} + \underbrace{k_{\text{IRLS}} \cdot O(H^3)}_{\text{iterations cost}}$$

For problems where $H \ll M$, the pre-computation dominates and IRLS is very efficient.

A cold start $\mathbf{w}_0 = \mathbf{0}$ saves the $O(H^3)$ OLS solve in the pre-computation but loses iterations to compensate, since the initial distance $\|\mathbf{w}_0 - \mathbf{w}^*\|$ multiplies the rate of Theorem 2.2. We use the warm start by default and quantify the trade-off in Section 5.3.

2.7 Expected behavior on our problem

On the ELM LASSO problem we expect $f(\mathbf{w}_k)$ to decrease at every iteration. Any non-decreasing step indicates an implementation error. On a semilog plot of $f(\mathbf{w}_k) - f^*$ against k the curve should be approximately straight, with a slope governed by the conditioning of $\mathbf{Q}_k$ and the safety threshold ε_{thr}.

Sparsity arises dynamically: components $(w_k)_i$ that start small receive larger weights $|(w_k)_i|^{-1}$ and are pushed further toward zero. At convergence many components lie inside the ε_{thr}-ball of zero. The algorithm has two hyperparameters: λ_{LASSO} (model) and ε_{thr} (numerical). Larger λ_{LASSO} gives sparser solutions and a higher training residual. It also enters $\mathbf{Q}_k$'s conditioning and so affects the rate. Smaller ε_{thr} approximates the L_1 term more tightly at the cost

of stability when a component approaches zero. Default: $\varepsilon_{\text{thr}} = 10^{-8}$ (calibrated in Section 5.4).

Chapter 3

Algorithm 2: Deflected Subgradient Method

In this chapter, we apply the deflected subgradient method with Polyak step and with target level (SGPTL) to the ELM LASSO problem.

3.1 Why the plain subgradient method is inadequate

The ELM LASSO problem (1.3) is nonsmooth due to the L_1 penalty. The plain subgradient method converges on convex nonsmooth problems but has two structural limitations on ELM LASSO. First, the update $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{g}_i$ is memoryless [10, Slide 10] and zig-zags on ill-conditioned $\mathbf{X}^T\mathbf{X}$: its iteration complexity is the optimal $\Omega(L^2/\varepsilon^2)$ [4, Theorem 3.2], with L scaling like $\|\mathbf{X}\|_2$ and the sublevel-set radius. Second, a pre-set diminishing–square-summable schedule $\sum_i \alpha_i = \infty$, $\sum_i \alpha_i^2 < \infty$ [10, Slide 10] forces the step to vanish regardless of the actual gap to f^*, so practical progress stalls. We address the first limitation with deflection (memory in the direction) and the second with an adaptive Polyak target-level stepsize (Section 3.3).

3.1.1 The deflection mechanism

Deflected subgradient methods [10, Slide 15] replace the raw subgradient with a direction that blends the current subgradient with the previous search direction:

$$\mathbf{d}_i = \gamma_i \, \mathbf{g}_i + (1 - \gamma_i) \, \mathbf{d}_{i-1} \quad (i \geq 1), \qquad \mathbf{d}_0 = \mathbf{g}_0, \tag{3.1}$$

where $\gamma_i \in (0, 1]$ is the deflection parameter. The update is $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \, \mathbf{d}_i$: $\gamma_i = 1$ recovers plain subgradient, $\gamma_i < 1$ damps the zig-zag. The method specifies γ_i as the minimiser of $\|\mathbf{d}_i\|^2$ on $[0, 1]$ [10, Slide 15], which maximises the Polyak step α_i since $\|\mathbf{d}_i\|^2$ is its denominator.

3.2 Convergence framework: balancing deflection and stepsize

We use the stepsize-restricted Polyak rule [10, Slide 15]:

$$\alpha_i = \frac{\beta_i \, (f(\mathbf{w}_i) - f^*)}{\|\mathbf{d}_i\|_2^2}, \quad \beta_i \le \gamma_i, \tag{3.2}$$

with $\beta_i = \min(\beta, \gamma_i)$ and $\beta = 1$ [10, Slide 14]. Since f^* is unknown, we substitute the target level $f_{\text{ref}} - \delta$ (Section 3.3).

Deflection floor $\gamma_{\min} > 0$. Condition (3.5) of [7] requires (when $\lambda_k \ge 0$) a uniform bound $\beta_k \ge \beta^* > 0$ on the stepsize multiplier, not pointwise positivity. The closed-form greedy (3.3) keeps $\gamma_i \in (0, 1]$ at each step but does not bound the sequence away from zero: when consecutive subgradients align near stationarity, the numerator $\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle$ of γ^* in (3.4) tends to 0^+, and with it β_i and the Polyak step. Projecting onto $[\gamma_{\min}, 1]$ supplies the missing uniform bound with $\beta^* = \gamma_{\min}$. The calibration of $\gamma_{\min}$ is in Section 3.4.1, the numerical sweep in Section 5.4.

3.2.1 Optimal deflection parameter

The argmin problem

$$\gamma_i \in \operatorname*{arg\,min}_{\gamma \in [\gamma_{\min}, 1]} \|\gamma \, \mathbf{g}_i + (1 - \gamma) \, \mathbf{d}_{i-1}\|_2^2 \tag{3.3}$$

is quadratic in γ: expanding $\|\gamma \mathbf{g}_i + (1-\gamma)\mathbf{d}_{i-1}\|^2$ by bilinearity and taking the vertex of the resulting upward parabola gives the closed form [10, Slide 15], with full derivation in Appendix D:

$$\gamma^* = \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2}, \quad \gamma_i = \min(1, \max(\gamma_{\min}, \gamma^*)). \tag{3.4}$$

The clip onto $[\gamma_{\min}, 1]$ instead of $[0, 1]$ is our addition (Section 3.2). The degenerate case $\mathbf{g}_i = \mathbf{d}_{i-1}$ has vanishing leading coefficient in the parabola and we set $\gamma_i = 1$ by convention. The first iteration is handled the same way: with $\mathbf{d}_{-1} = \mathbf{0}$, (3.4) returns $\gamma^* = 0$. We override $\gamma_0 = 1$ so that $\mathbf{d}_0 = \mathbf{g}_0$.

3.3 The target-level mechanism

The Polyak stepsize [10, Slide 12] requires f^*. The fundamental inequality [10, Slide 10] is

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \le \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - 2\alpha_i (f(\mathbf{w}_i) - f^*) + (\alpha_i)^2 \, \|\mathbf{g}_i\|_2^2 , \tag{3.5}$$

minimized at $\alpha_i^* = (f(\mathbf{w}_i) - f^*)/\|\mathbf{g}_i\|_2^2$ [10, Slide 12]. SGPTL replaces f^* with the target level $f_{\text{ref}} - \delta$ [10, Slide 14]: f_{ref} is the best value found so far, $\delta > 0$ an estimate of the residual gap.

A failure mode of this substitution is $f_{\text{ref}} - \delta < f^*$, in which case $f(\mathbf{w}_{i+1}) \geq f^* > f_{\text{ref}} - \delta/2$ for every iterate and the sufficient-descent test $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$ is unsatisfiable: f_{ref} never updates and the algorithm silently stalls. The mechanism guards against this through an internal patience counter $r_i = \sum \alpha_j \|\mathbf{d}_j\|$, the displacement accumulated since the last f_{ref} update. When r_i exceeds a threshold R without sufficient descent firing, δ is contracted to $\rho\delta$ with $\rho \in (0, 1)$ and r is reset. Repeated contractions drive $\delta \to 0$, eventually restoring $f_{\text{ref}} - \delta \geq f^*$ and unblocking the sufficient-descent test.

3.4 The complete algorithm

Algorithm 2 presents the full Deflected Subgradient Method with target level for ELM LASSO.

Numerical safeguards. Three pure floating-point guards lie outside the pseudocode.

1. If $\|\mathbf{d}_i\|^2 < 10^{-30}$ we terminate.
2. If $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta) \leq 0$ we set $\alpha_i = 0$ (condition (3.5) of [7]) and trigger the sufficient-descent update at $\mathbf{w}_i$, since $f(\mathbf{w}_i) \leq f_{\text{ref}} - \delta/2$.
3. If $\mathbf{w}_{i+1}$ has a non-finite component we skip the iteration without touching $(\delta, f_{\text{ref}}, r)$.

3.4.1 Parameter calibration for ELM LASSO

SGPTL has five parameters: β, $\gamma_{\min}$, δ_0, ρ, R. Each is discussed below with the choice we adopt for ELM LASSO and its rationale.

Cold vs. warm start. Theorem 3.1 places no condition on $\mathbf{w}_0$. We adopt the default cold start $\mathbf{w}_0 = \mathbf{0}$ and additionally support the OLS warm start $(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$, which lets us run IRLS and SGPTL from the same point when comparing them (Chapter 4). Per-instance behaviour is in Section 5.3.

$\beta = 1$. [7, (3.1)] admits $\beta_k \in (0, 1]$. Substituting the Polyak step $\alpha_i = \beta\,(f(\mathbf{w}_i) - f^*)/\|\mathbf{g}_i\|_2^2$ into the fundamental inequality (3.5) gives a per-iteration reduction in $\|\mathbf{w}_i - \mathbf{w}^*\|_2^2$ proportional to $\beta(2 - \beta)$, a downward parabola maximised at $\beta = 1$: the deflected variant of [10, Slide 15] inherits the property. We fix $\beta = 1$ so that $\beta_i = \min(\beta, \gamma_i)$ is driven by γ_i.

Algorithm 2 Deflected Subgradient with Target Level (SGPTL) for ELM LASSO [10, Slide 15]

Require: f (ELM LASSO objective); $i_{\max}$; $\beta \in (0, 1]$; $\delta_0 > 0$; $R > 0$; $\rho \in (0, 1)$; $\gamma_{\min} \in (0, 1]$

$\mathbf{w}_0 \leftarrow \mathbf{0}$ *(cold start by default, warm start $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ is admissible. See §3.6)*
$r \leftarrow 0$; $\delta \leftarrow \delta_0$; $f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(\mathbf{w}_0)$
for $i = 0, 1, \ldots, i_{\max}$ **do**
    Compute $\mathbf{g} \in \partial f(\mathbf{w}_i)$ *(subgradient of LASSO)*
    if $i = 0$ **then** *(no $\mathbf{d}_{-1}$ available)*
        $\gamma_i \leftarrow 1$; $\mathbf{d}_i \leftarrow \mathbf{g}$
    else
        Compute γ_i via (3.4) *(optimal deflection, clipped to $[\gamma_{\min}, 1]$)*
        $\mathbf{d}_i \leftarrow \gamma_i \mathbf{g} + (1 - \gamma_i)\mathbf{d}_{i-1}$ *(deflected direction)*
    end if
    $\beta_i \leftarrow \min(\beta, \gamma_i)$ *(stepsize-restricted, $\beta = 1$ default. See §3.2)*
    $\alpha_i \leftarrow \beta_i (f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)) / \|\mathbf{d}_i\|_2^2$ *(Polyak step)*
    $\mathbf{w}_{i+1} \leftarrow \mathbf{w}_i - \alpha_i \mathbf{d}_i$ *(update)*
    $\bar{f} \leftarrow \min(\bar{f}, f(\mathbf{w}_{i+1}))$ *(update record)*
    if $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$ **then** *(sufficient descent)*
        $f_{\text{ref}} \leftarrow \bar{f}$; $r \leftarrow 0$
    else if $r > R$ **then** *(travel-distance patience exhausted)*
        $\delta \leftarrow \delta \rho$; $r \leftarrow 0$
    else
        $r \leftarrow r + \alpha_i \|\mathbf{d}_i\|_2$
    end if
end for
return $\mathbf{w}_{\text{best}}$ (iterate achieving $\bar{f}$)

$\gamma_{\min} = 0.05$**.** The value inside $(0, 1]$ trades off two effects: small $\gamma_{\min}$ leaves the clip almost inactive (the deflected direction stays close to the unconstrained greedy minimiser of (3.4) and the Polyak step keeps its locally optimal magnitude) but multiplies the rate of Theorem 3.1 by $1/\gamma_{\min}$ (so the iteration count by $1/\gamma_{\min}^2$), large $\gamma_{\min}$ shrinks the constant but binds the clip and weakens the memory term in $\mathbf{d}_i$. We adopt $\gamma_{\min} = 0.05$. Numerical justification in Section 5.4.

$\delta_0 = c\, f(\mathbf{w}_0)$ **with** $c = 0.1$**.** [7, Lemma 3.8] requires only $\delta_0 > 0$: we need a problem-dependent scale that does not presuppose f^*. We scale on the objective at the starting point, $\delta_0 = c\, f(\mathbf{w}_0)$. Since f^* is the global minimum, $f(\mathbf{w}_0) \geq f^*$ for any $\mathbf{w}_0$, so the initial target $f_{\text{ref}} - \delta_0$ is a valid overestimate of the residual gap: if it overshoots, the patience mechanism of Section 3.3 contracts δ until the target is feasible. For the default cold start $\mathbf{w}_0 = \mathbf{0}$ this scale is $f(\mathbf{0}) = \frac{1}{2}\|\mathbf{y}\|_2^2$, an $O(M)$ evaluation that needs no precomputation. A warm start instead pays the $O(MH^2 + H^3)$ to form $\mathbf{w}_{\text{OLS}}$ first.

The multiplier c trades off two failure modes: small c keeps the target $f_{\text{ref}} - \delta$ close to f_{ref} and δ barely contracts, large c pushes the target below f^* so that only the patience trigger drives progress. Following the moderate values of the target-level literature [3, 13, 1] we set $c = 0.1$. All experiments in Chapter 5 use this admissible scale $\delta_0 = c\, f(\mathbf{w}_0)$, the f^*-based scale $\delta_0 = c\, f^*$ that appears in Figure A.1 is reported only as an abstract sensitivity reference and is never fed to the optimizer, since f^* is unknown in practice.

$\rho = 0.7$**.** [7, Lemma 3.8] admits any $\rho \in (0, 1)$. The target-level literature [3, 13, 1], [10, Slide 14] recommends the moderate range [0.5, 0.7]: too close to 1 contracts δ slowly and keeps the target aggressive, too close to 0 contracts before each target level has been exploited. We adopt $\rho = 0.7$. Sensitivity sweep in Section 5.4.

$R = 1$**.** [7, Lemma 3.8] admits any $R > 0$ and gives no specific value. Tradeoff: R too small contracts δ on every minor stagnation, wasting potential progress between target updates, R too large makes the sufficient-descent test the sole driver and stalls if the target is structurally infeasible. R should therefore sit on the per-iteration travel scale $\alpha_i \|\mathbf{d}_i\|$, which under our other defaults on ELM LASSO with column-normalised $\mathbf{X}$ is of order 10^{-3}. We adopt $R = 1$, so the travel-distance branch fires every $\mathcal{O}(10^3)$ stalled iterations. Section 5.3 reports the empirical contraction counts.

3.5 Application to the ELM LASSO problem

3.5.1 Subgradient computation for ELM LASSO

Section 1.4.3 already gives the subdifferential of the ELM LASSO objective. Algorithm 2 requires a single element $\mathbf{g}_i \in \partial f(\mathbf{w}_i)$ per iteration: we pick the

canonical sign-based representative

$$\mathbf{g} = \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}}\,\mathbf{s}, \qquad s_i = \operatorname{sign}(w_i) \in \partial|w_i|, \tag{3.6}$$

with the convention $\operatorname{sign}(0) = 0$, which is admissible since $0 \in [-1, 1] = \partial|0|$. This choice is not the minimum-norm element of $\partial f(\mathbf{w}_i)$, but Theorem 3.1 only requires admissibility. The event $(\mathbf{w}_i)_j = 0$ exactly has probability zero for $i \geq 1$: the update $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{d}_i$ depends continuously on the past iterates and on the continuous δ-contraction sequence. The cold start $\mathbf{w}_0 = \mathbf{0}$ is the only iterate where the convention is exercised, and only at $i = 0$.

3.5.2 Convergence analysis and hypothesis verification for ELM LASSO

Theorem 3.1 (Convergence of SGPTL, from [7])**.** *Let $f : \mathbb{R}^H \to \mathbb{R}$ be convex with $f^* = \min_{\mathbf{w}} f(\mathbf{w}) > -\infty$ attained at some $\mathbf{w}^*$, and let $\{\mathbf{w}_i\}$ be the sequence generated by Algorithm 2. Assume the subgradients encountered along the run are uniformly bounded, $\|\mathbf{g}_i\|_2 \leq L$, and the stepsize-restricted rule $\beta_i \leq \gamma_i$ holds with $\gamma_i \geq \gamma_{\min} > 0$. Then the record values $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h)$ converge to f^*, and once the target-level estimate has sharpened to f^* (i.e. $\delta_k \to 0$ and $f^k_{\text{ref}} \to f^*$, which Lemma 3.8 of [7] guarantees) the asymptotic rate, derived below from eq. (3.17) in the proof of Theorem 3.5 of [7], is*

$$\bar{f}_k - f^* \;\leq\; \frac{L\,\|\mathbf{w}_0 - \mathbf{w}^*\|_2}{\gamma_{\min}\sqrt{k+1}}\,, \tag{3.7}$$

so the worst-case complexity to attain $\bar{f}_k - f^* \leq \varepsilon$ is $O\big(L^2/(\gamma_{\min}^2\,\varepsilon^2)\big)$.

Proof. We verify the hypotheses of [7, Theorem 3.7] on Algorithm 2 and read off the conclusions. The framework of [7] is stated for σ_k-inexact subgradients, where $\sigma_k \geq 0$ bounds the oracle error. We compute $\mathbf{g}_i$ exactly via (3.6), so the inexactness parameter is zero. The remaining notation maps to ours via $\alpha_k \leftrightarrow \gamma_i$, $\beta_k \leftrightarrow \beta_i$, $\nu_k \leftrightarrow \alpha_i$, $\mu \leftrightarrow \rho$, $X = \mathbb{R}^H$ (no constraint), and $\lambda_k \leftrightarrow f(\mathbf{w}_i) - (f^k_{\text{ref}} - \delta_k)$.

(a) Deflection-consistency condition (2.13). The condition of [7, Lemma 2.4] requires $v_{k+1} = \tilde{d}_k$ whenever $d_k = \tilde{d}_k$, where $\tilde{d}_k$ is the unprojected deflected direction. In our unconstrained setting $X = \mathbb{R}^H$ no projection occurs, so $d_k = \tilde{d}_k$ at every iteration and Algorithm 2 sets $v_{k+1} = d_k$, so (2.13) holds trivially.

(b) Stepsize condition (3.5) of [7]. Translated through the notation mapping above ($\alpha_k \leftrightarrow \gamma_i$, $\beta_k \leftrightarrow \beta_i$), the condition has two clauses:

- if $\lambda_k \geq 0$ then $\gamma_i \geq \beta_i \geq \beta^* > 0$ for some uniform constant $\beta^* > 0$ independent of i,
- if $\lambda_k < 0$ then the step is null, $\alpha_i = 0$.

Case $\lambda_k > 0$ (target not yet reached). Line 12 of Algorithm 2 sets $\beta_i = \min(1, \gamma_i)$, and the clip (3.4) keeps $\gamma_i \in [\gamma_{\min}, 1]$. Hence $\beta_i \geq \gamma_{\min}$ for every i,

and the uniform lower bound is satisfied with $\beta^* = \gamma_{\min}$. The other inequality $\gamma_i \geq \beta_i$ is just $\gamma_i \geq \min(1, \gamma_i)$, which holds by definition of min.

Case $\lambda_k \leq 0$ (target already attained at $\mathbf{w}_i$). The numerical safeguard of Section 3.4 skips the iteration with $\alpha_i = 0$, which is the second clause verbatim.

(c) Target-level threshold condition (3.26) of [7]. The condition requires *either* $f_{\text{ref}}^{\infty} = f^* = -\infty$ *or* both $\liminf_{k\to\infty} \delta_k = 0$ and the series condition (3.25) of [7], $\sum_k \lambda_k / \|\mathbf{d}_k\|^2 = \infty$. Since on ELM LASSO $f^* > -\infty$, the first branch is excluded and we need the second. [7, Lemma 3.8] provides a concrete update strategy for $(f_{\text{ref}}^k, \delta_k)$ — the sufficient-descent test on $f_{k+1} \leq f_{\text{ref}}^k - \delta_k/2$, the target-infeasibility test on $r_k > R$, and the accumulate-r counter — that delivers both $\liminf_{k\to\infty} \delta_k = 0$ and (3.25). Lines 16–21 of Algorithm 2 implement that strategy verbatim under the identification $\mu \leftrightarrow \rho$, so condition (3.26) holds.

Subgradients uniformly bounded. The convex objective f of ELM LASSO is coercive and continuous. The iterates $\{\mathbf{w}_i\}$ remain in a compact sublevel set S_{c_0} for some $c_0 \geq f(\mathbf{w}_0)$, and ∂f is uniformly bounded on S_{c_0} by Lipschitz continuity of convex functions on compact sets [11, Slide 9].

Conditions (2.13), (3.5), (3.26) of [7] being verified with $\sigma^* = 0$, [7, Theorem 3.7] gives $f_{\text{ref}}^{\infty} \leq f^* + \sigma^* = f^*$, so $\bar{f}_i \to f^*$. To attach a rate to this convergence we work in the regime where the target-level update has driven $f_{\text{ref}}^k - \delta_k$ to f^* (Lemma 3.8 of [7]). The Polyak step at iteration i then uses target f^* exactly, $\alpha_i = \beta_i (f(\mathbf{w}_i) - f^*) / \|\mathbf{d}_i\|_2^2$.

Per-step descent. Specialising eq. (3.17) of [7] (the per-step inequality used inside the proof of their Theorem 3.5) to the exact-oracle case $\sigma_k = 0$ with the inner correction γ_k of (3.1) set to zero and $\bar{x} = \mathbf{w}^*$ gives

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \;\leq\; \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 \;-\; \beta_i \,(2\gamma_i - \beta_i)\, \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2}. \tag{3.8}$$

The full derivation (identity (2.9) + Corollary 3.2 + substitution of the Polyak step) is in Appendix C. Algorithm 2 sets $\beta_i = \min(1, \gamma_i) = \gamma_i$ since $\gamma_i \leq 1$, so

$$\beta_i \,(2\gamma_i - \beta_i) \;=\; \gamma_i \,(2\gamma_i - \gamma_i) \;=\; \gamma_i^2 \;\geq\; \gamma_{\min}^2.$$

Telescoping. Summing (3.8) from $i = 0$ to $i = k$ and dropping the non-negative left-hand side at index $k + 1$,

$$\gamma_{\min}^2 \sum_{i=0}^{k} \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2} \;\leq\; \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2 \,.$$

Each $\mathbf{d}_i$ is, by induction on the recurrence (3.1), a convex combination of subgradients along the run, so $\|\mathbf{d}_i\|_2 \leq \max_{h \leq i} \|\mathbf{g}_h\|_2 \leq L$. Substituting,

$$\sum_{i=0}^{k} (f(\mathbf{w}_i) - f^*)^2 \;\leq\; \frac{L^2 \,\|\mathbf{w}_0 - \mathbf{w}^*\|_2^2}{\gamma_{\min}^2}.$$

The record value satisfies $\bar{f}_k - f^* = \min_{h \le k}(f(\mathbf{w}_h) - f^*) \le f(\mathbf{w}_i) - f^*$ for every $i \le k$, so $(k+1)(\bar{f}_k - f^*)^2$ lower-bounds the sum, giving (3.7).

Iteration count. Solving the right-hand side of (3.7) for ε yields

$$k + 1 \;\ge\; \frac{L^2 \, \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2}{\gamma_{\min}^2 \, \varepsilon^2}.$$

The non-deflected Polyak bound under the same hypotheses (same derivation with the per-step factor $\beta_i(2 - \beta_i)$ in place of $\beta_i(2\gamma_i - \beta_i)$, attainable when there is no deflection clip, i.e. $\alpha_k \equiv 1$ in [7]) is $k + 1 \ge L^2 \, \|\mathbf{w}_0 - \mathbf{w}^*\|_2^2 / \varepsilon^2$, so the deflection floor inflates the iteration count by the factor $1/\gamma_{\min}^2$. With $\gamma_{\min} = 0.05$ this is a $400\times$ multiplier. The rate order $O(L^2/\varepsilon^2)$ is unchanged. □

Verification of theorem assumptions for ELM LASSO. Coercivity of f (Proposition 1.1) makes every sublevel set $\{f \le c\}$ compact, so $\{\mathbf{w}_i\}$ stays in a bounded set and $f^* = \min f > -\infty$ is attained, which gives the theorem's first hypothesis. The subgradient bound used in the proof is local: the smooth part $\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ grows with $\|\mathbf{w}\|$, but on the compact sublevel set it is Lipschitz, so L in (3.7) is finite. The stepsize-restricted clause $\beta_i \le \gamma_i$ holds by construction via $\beta_i = \min(1, \gamma_i)$, and $\gamma_i \ge \gamma_{\min}$ by the floor.

Complexity. The lower bound for first-order methods on nonsmooth convex problems is $\Omega(L^2/\varepsilon^2)$ [10, Slide 8], [4, Theorem 3.13]. Algorithm 2 attains it in the order $O(L^2/\varepsilon^2)$ (Theorem 3.1).

3.6 Expected practical behavior

Theorem 3.1 gives $\bar{f}_k - f^* \lesssim L \, \|\mathbf{w}_0 - \mathbf{w}^*\|_2 / (\gamma_{\min} \sqrt{k+1})$. On a semilog plot of $\bar{f}_i - f^*$ we therefore expect a concave curve closing as $1/\sqrt{i}$, with $f(\mathbf{w}_i)$ oscillating around $\bar{f}_i$ because the theorem bounds the record, not the raw value.

The bound depends on two problem-specific quantities. L scales with $\|\mathbf{X}\|_2$ and λ_{LASSO}, so larger λ_{LASSO} shifts the envelope up. The initial distance $\|\mathbf{w}_0 - \mathbf{w}^*\|_2$ enters multiplicatively and separates the two starts: cold gives $\|\mathbf{w}^*\|_2$, OLS gives $\|\mathbf{w}_{\text{OLS}} - \mathbf{w}^*\|_2$, which is small whenever OLS sits near the LASSO minimiser (the two coincide at $\lambda_{\text{LASSO}} = 0$). Whether the warm start improves the bound on a given instance depends on $\|\mathbf{w}_{\text{OLS}} - \mathbf{w}^*\|_2$ relative to $\|\mathbf{w}^*\|_2$. Per-instance numbers in Section 5.3.

The rate controls $\bar{f}_i - f^*$, not individual coordinates of $\mathbf{w}_i$. A subgradient step updates each coordinate continuously and never enforces $(\mathbf{w}_i)_j = 0$ exactly. The residual magnitude on inactive components is bounded by the rate scale $O(L/(\gamma_{\min}\sqrt{i}))$. Hard sparsity therefore requires a thresholding post-step above this scale. Termination by $\|\mathbf{g}_i\|_2 \to 0$ is similarly unreliable: $0 \in \partial f(\mathbf{w}^*)$ holds at $\mathbf{w}^*$ only, not at finite-budget iterates, so we rely on $i_{\max}$ or a flat-window test on $\bar{f}_i$.

Chapter 4

Algorithm Comparison

In this chapter, we will compare the two algorithms together to analyze their properties and trade-offs.

4.1 Core strategy: how each algorithm handles non-smoothness

IRLS: smoothing via majorization
IRLS replaces f with a smooth quadratic majorant $Q(\mathbf{w}, \mathbf{w}_k)$ (2.2), reducing each step to a weighted L_2 problem solved in closed form via the normal equations (2.3).

DSM: direct subgradient with memory
DSM descends along a subgradient $\mathbf{g} \in \partial f(\mathbf{w}_i)$ (3.5.1), deflected with the previous direction (3.1.1) with Polyak step using a self-adapting target estimate of f^* (3.3).

4.2 Convergence behaviour

4.2.1 Monotonicity

The majorization-minimization construction of Section 2.2 guarantees IRLS is **monotone**: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$. DSM is **non-monotone**: only the record $\bar{f}^{\,i} = \min_{h \leq i} f(\mathbf{w}_h)$ is monotone, so DSM returns $\mathbf{w}_{\text{best}}$ and stops on a fixed budget.

4.2.2 Rate

IRLS achieves a **linear** rate (Section 2.5), DSM achieves the optimal **sublinear** $O(L^2/\varepsilon^2)$ rate for nonsmooth convex functions [10, Slide 8] (Theorem 3.1). Lowering ε:

- DSM: $\bar{f}_k - f^* \lesssim L \left\|\mathbf{w}_0 - \mathbf{w}^*\right\|_2 / (\gamma_{\min} \sqrt{k+1})$ gives $k \propto 1/(\gamma_{\min}^2 \varepsilon^2)$. Lowering ε by a factor 10 therefore multiplies the iteration count by $10^2 = 100$ (the $\gamma_{\min}$ floor inflates the constant but not the per-decade cost).
- IRLS: $f(\mathbf{w}_{k+1}) - f^* \leq r\,(f(\mathbf{w}_k) - f^*)$ with $r \in (0,1)$ (Theorem 2.2) gives $k \propto \log(1/\varepsilon)/\log(1/r)$. Lowering ε by a factor 10 *adds* the constant $\log 10/\log(1/r)$ iterations, independent of ε.

4.2.3 Convergence on the ELM problem

By Proposition 1.1, $\mathbf{X}$ full column rank makes f μ-strongly convex with $\mu = \lambda_{\min}(\mathbf{X}^\top \mathbf{X}) > 0$, with a unique minimizer $\mathbf{w}^*$. Both algorithms are theoretically guaranteed to converge on this problem, but with different notions of convergence: Theorem 2.2 gives IRLS iterates $\mathbf{w}_k \to \mathbf{w}^*$ at a linear rate, while Theorem 3.1 gives SGPTL only the weaker guarantee $\bar{f}_i \to f^*$ on the record sequence $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h)$, but single iterates $\{\mathbf{w}_i\}$ may oscillate.

4.3 Computational cost

4.3.1 Per-iteration cost

Operation	**IRLS**	**DSM**
Weight / direction update	$O(H)$	$O(H)$
Matrix-vector product(s)	—	$O(MH)$
Linear system solve	$O(H^3)$ (Cholesky) or $O(pH^2)$ (CG)	—
Total per iteration	$O(H^3)$	$O(MH)$

Table 4.1: Per-iteration computational cost of IRLS and DSM.

IRLS solves an $H \times H$ SPD system per iteration (2.3): $O(H^3/3)$ via Cholesky, or $O(pH^2)$ via p CG steps when H is large and $\mathbf{Q}_k$ well-conditioned.

DSM uses two matrix-vector products per iteration ($\mathbf{X}\mathbf{w}_i$ and $\mathbf{X}^T(\cdot)$, each $O(MH)$) (3.5.1).

4.3.2 Total cost

Accounting for both the precomputation phase and the iteration loop, the total costs are

$$\begin{aligned}
\text{IRLS (OLS warm start):} &\quad O(MH^2) + O(MH) + O(H^3) + k_{\text{IRLS}} \cdot O(H^3) \\
\text{IRLS (cold start):} &\quad O(MH^2) + O(MH) + k_{\text{IRLS}} \cdot O(H^3) \\
\text{SGPTL with OLS warm start:} &\quad O(MH^2) + O(H^3) + k_{\text{DSM}} \cdot O(MH), \\
\text{SGPTL with cold start } \mathbf{w}_0 = \mathbf{0}\text{:} &\quad k_{\text{DSM}} \cdot O(MH).
\end{aligned}$$

On the ELM problem, $M \gg H$ is typical, so $O(MH^2)$ dominates IRLS's total. IRLS therefore wins whenever $k_{\mathrm{DSM}} \gtrsim H$: the warm-start variant of SGPTL already pays the same $O(MH^2)$ overhead and adds the iteration cost on top, while the cold-start variant trades the overhead for k_{DSM} matrix-vector products at $O(MH)$ each. Since Theorem 3.1 forces $k_{\mathrm{DSM}} = O(L^2/\varepsilon^2)$ with L growing in H (the ℓ_1 subgradient has norm $\leq \lambda_{\mathrm{LASSO}}\sqrt{H}$), the inequality $k_{\mathrm{DSM}} > H$ holds at every H we test for any reasonable accuracy.

4.4 Solution quality

4.4.1 Sparsity

IRLS yields numerically sparse iterates whenever we have small components of $\mathbf{w}$, which are sent towards ε (theoretically 0) (2.7). DSM gives only **approximate sparsity**: an explicit thresholding step is needed (3.6).

4.4.2 Accuracy

At equal budget, IRLS reaches much higher accuracy on small-to-moderate problems thanks to its linear rate. DSM is competitive only when $H^2 \gg M$ (Table 4.1).

4.5 Comparison summary for the ELM problem

Criterion	**IRLS**	**DSM**
Approach	Smoothing (MM)	Direct subgradient
Monotone	Yes	No (record value only)
Convergence rate	Linear	Sublinear $O(L^2/\varepsilon^2)$
Per-iteration cost	$O(H^3)$	$O(MH)$
Precomputation	$O(MH^2) + O(MH) + O(H^3)$	None
Exact sparsity	Yes	No (needs thresholding)
Parameters to tune	$\varepsilon_{\mathrm{thr}}$	δ_0, ρ, R, β
Stopping criterion	Reliable (iterate change)	Unreliable (fixed budget)
Preferred when	High accuracy, $H \ll M$	Large H, moderate accuracy

Table 4.2: Summary comparison of IRLS and DSM on the ELM LASSO problem.

Making the assumption that $M \gg H$, IRLS is preferred: the $O(MH^2)$ precomputation of $\mathbf{A}$ is paid back over a few cheap Cholesky solves that produce sparse iterates, with $\varepsilon_{\mathrm{thr}}$ the only tunable parameter. DSM is the better choice only when $H^2 \gg M$ or $\mathbf{X}^T\mathbf{X}$ does not fit in memory, and the accuracy target is moderate ($\varepsilon \sim 10^{-3}$).

Chapter 5

Experimental Results

This chapter tests the IRLS and SGPTL implementations against the predictions of Chapters 2–3 (convergence behaviour on ELM LASSO) and against the cost–accuracy comparison of Chapter 4.

5.1 Experimental setup and datasets

Implementation. The code under `progetto/code/src/` is split into five modules: `lasso_utils.py` (objective, smooth-part gradient, subgradient selection), `linear_solvers.py` (Cholesky via `scipy.linalg.cho_factor/cho_solve` and a hand-rolled Jacobi-preconditioned CG), `irls.py` (Algorithm 1), `deflected_subgradient.py` (Algorithm 2), and `elm.py` (random hidden layer). Core algorithms are written from scratch (reusing only NumPy/SciPy as primitives). A pytest suite cross-checks both algorithms against `sklearn.linear_model.Lasso` and verifies the surrogate-descent and record-monotonicity invariants iterate by iterate.

Reference solution. We obtain $\mathbf{w}^*$ from `sklearn.linear_model.Lasso` (coordinate descent, `tol`$= 10^{-12}$, 10^5 iterations, `fit_intercept=False`). Sklearn's loss has a $1/(2M)$ prefactor on the squared residual and weights the ℓ_1 term by α, so the equivalent of (1.3) is obtained by passing $\alpha_{\text{sklearn}} = \lambda_{\text{LASSO}}/M$; we then evaluate $f^* = f(\mathbf{w}^*)$ from (1.3) directly. The returned $\mathbf{w}^*$ satisfies the optimality condition (1.5) below 10^{-3} on every instance. On the ELM-transformed real data, where coordinate descent does not converge in a realistic budget, we replace this reference with the IRLS-converged value cross-validated against CVXPY's interior-point solver Clarabel (Section 5.6).

5.1.1 Datasets

We use three instances spanning two conditioning regimes.

Synthetic LASSO benchmark (`src/data_generation.py`). We sample a sparse $\mathbf{w}_{\text{true}} \in \mathbb{R}^H$ (10–15% non-zero, entries $\sim \mathcal{N}(0,1)$), draw $\mathbf{X} \in \mathbb{R}^{M \times H}$ with i.i.d. Gaussian columns normalised to unit ℓ_2 norm, and set $\mathbf{y} = \mathbf{X}\mathbf{w}_{\text{true}} + \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, 0.05^2\mathbf{I})$. This is the benign regime for LASSO. We pick $M \geq 2H$ to keep $\kappa(\mathbf{X}^\top\mathbf{X})$ at a moderate constant by random-matrix concentration [14], the planted $\mathbf{w}_{\text{true}}$ provides the ground truth for support-recovery metrics, and the noise std 0.05 puts the SNR in the moderate-noise regime (about 5–10% of the signal magnitude). It validates the algorithms in a well-conditioned setting but does *not* reproduce the conditioning of ELM data.

ELM-transformed real datasets (`diabetes`, `california`). We load `diabetes` ($n = 442$, $d = 10$) [9] and `california_housing` ($n = 20640$, $d = 8$) [17] from scikit-learn, split 80/20, standardise features and target on the train split, and apply the ELM map $\mathbf{X}_{\text{hidden}} = \sigma(\mathbf{X}\,\mathbf{W}_1^\top)$ with H sigmoid units and a fixed random $\mathbf{W}_1$. We use $H = 200$ for the main experiments and $H = 50$ where an independent solver (`cvxpy.CLARABEL`) is affordable for cross-validation. The sigmoid breaks the unit-norm column property and inflates $\kappa(\mathbf{X}_{\text{hidden}}^\top\mathbf{X}_{\text{hidden}})$ to $\sim 10^6$ on both datasets, orders of magnitude above the synthetic design. This is the ill-conditioned regime targeted by the experiments.

Limitations. (i) The synthetic benchmark lives in the well-conditioned, incoherent regime by construction, so positive results there do not extrapolate to ill-conditioned designs. (ii) The real datasets have no ground-truth support, so the *support-recovery* metrics of Section 5.5.1 (precision/recall/F_1 against the planted support) apply only to the synthetic instance; sparsity counts at a fixed threshold remain well-defined and are reported on real data alongside training-objective gaps and test MSE. (iii) On `california` at $\lambda_{\text{LASSO}} = 0.1$ the OLS warm start already sits very close to f^* (Section 5.3); we account for this in the warm-vs-cold and real-data comparisons.

5.2 Convergence and agreement with theory

Setting. Synthetic convergence instance, $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, sparsity 10%. Both algorithms are run from the same OLS warm start $\mathbf{w}_0 = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ so that any difference in the curves is attributable to the algorithm and not to the initialisation; the one-time Cholesky cost of forming $\mathbf{w}_0$ (Section 3.4.1) is charged to both wall-clock totals. The relevant objective values of this instance are $f(\mathbf{0}) \approx 76$ at the origin, $f(\mathbf{w}_0) \approx 1.56$ at the warm start, and $f^* \approx 1.13$ at the sklearn reference, which puts the initial gap $f(\mathbf{w}_0) - f^* \approx 0.43$ (the y-intercept of Figure 5.1).

IRLS: linear rate (Theorem 2.2). The expected behaviour from Chapter 2 is linear convergence of the MM iteration. Figure 5.1 (left) confirms it: after a fast transient the gap decreases at a constant per-iteration factor ≈ 0.848 in the asymptotic region, reaching $1.5 \cdot 10^{-6}$ in 100 iterations (iterate change $5.6 \cdot 10^{-6}$). The late flattening toward $\sim 10^{-6}$ is the accuracy floor of the smoothed surrogate at $\varepsilon_{\text{thr}} = 10^{-8}$ (Section 5.4), not a breakdown of the rate.

SGPTL: sublinear rate (Theorem 3.1). The expected behaviour from Chapter 3 is an $O(1/\sqrt{i})$ record gap and a non-monotone current value. Figure 5.1 (right, 8000 iterations, $\beta = 1$, $\gamma_{\min} = 0.05$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$, $\rho = 0.7$) confirms both: the record $\bar{f}^i - f^*$ follows the sublinear envelope down to $4.8 \cdot 10^{-5}$, and each decade of accuracy costs a $\sim 100\times$ increase in iterations, exactly the $\varepsilon \mapsto \varepsilon/10 \Rightarrow i \mapsto 100\, i$ scaling of the $O(L^2/\varepsilon^2)$ bound. The current value oscillates around the record, as the theory allows (the bound controls the record, not the raw iterate).

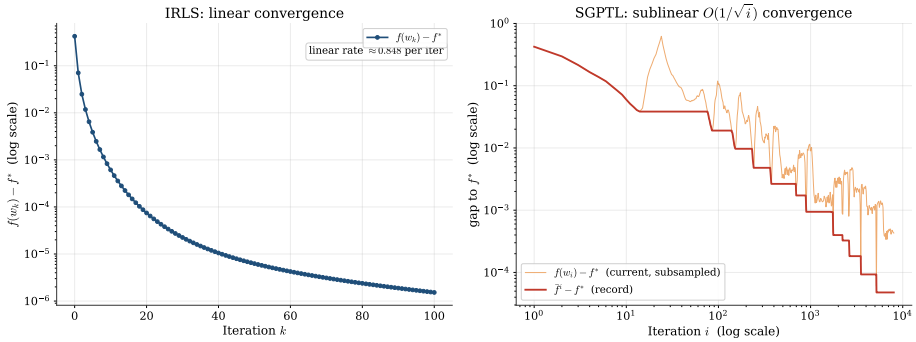

Figure 5.1: Optimality gap versus iteration on $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$. Left: IRLS, semilog axis; the annotation is the empirical per-iteration factor in the linear region. Right: SGPTL, log–log axes, current value (orange, subsampled log-uniformly) over the record (red). The record traces the $O(1/\sqrt{i})$ envelope of Theorem 3.1.

Against wall-clock time (Figure 5.2) SGPTL's cheaper iteration ($O(MH)$ vs $O(H^3)$) does not offset the slower rate at this size: IRLS reaches 10^{-6} in ~ 3 ms, SGPTL drops below 10^{-2} within ~ 10 ms and then crawls to $\sim 5 \cdot 10^{-5}$ over ~ 150 ms.

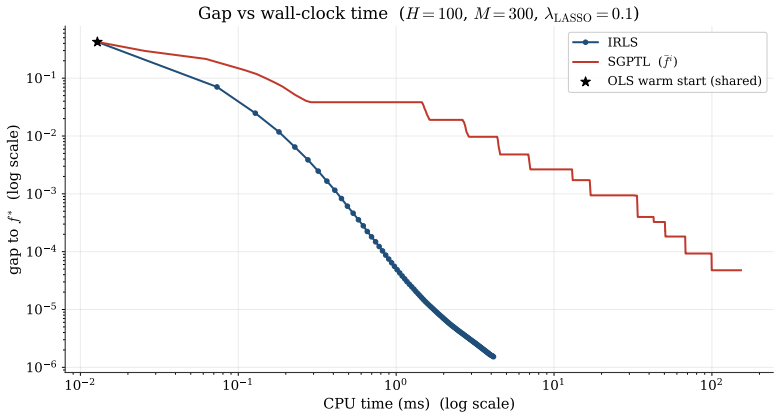

Figure 5.2: Optimality gap versus CPU time on the instance of Figure 5.1, log–log axes.

Both convergence curves above track the SGPTL *record* $\bar{f}^{\,i} = \min_{h \le i} f(\mathbf{w}_h)$ rather than the raw current value $f(\mathbf{w}_i)$: as anticipated in Section 3.3, the subgradient dynamics are not monotone, and only the record carries the rate guarantee of Theorem 3.1. Figure 5.3 makes this explicit: the current gap spikes well above the record at several points along the run, while the record is monotone by construction. This is also the practical reason the algorithm tracks and returns $\mathbf{w}_{\text{best}}$ rather than the last iterate.

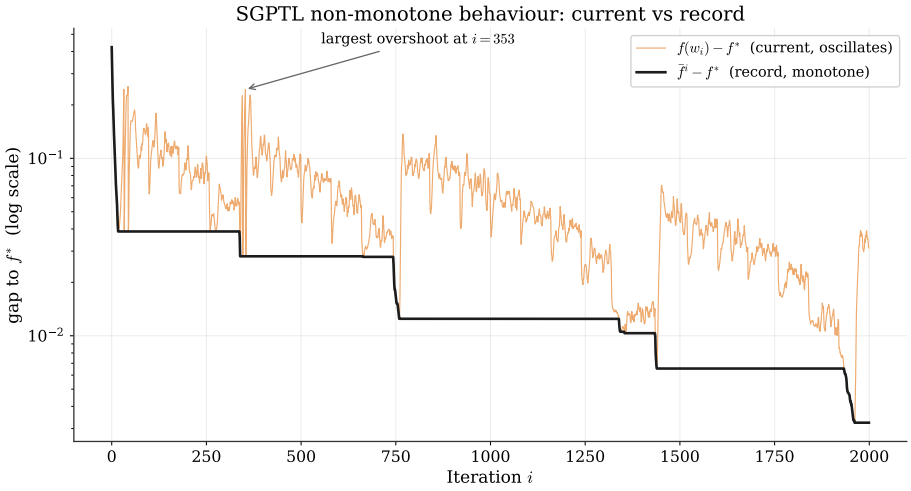

Figure 5.3: SGPTL on the convergence instance, first 2000 iterations, semilog y. The current gap (orange) overshoots the record (black) repeatedly; the largest overshoot is annotated.

5.3 Cold versus warm start

Setting. We run *both* algorithms from *both* initialisations on all three instances: the synthetic benchmark ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$) and the two ELM-transformed real datasets ($H = 200$). Both Theorem 2.2 (IRLS) and Theorem 3.1 (SGPTL) admit any $\mathbf{w}_0$; what changes between the starts is the initial distance $\|\mathbf{w}_0 - \mathbf{w}^*\|$, which enters their convergence bounds multiplicatively. Table 5.1 collects the final gaps; the rest of the section interprets them.

algorithm	start	synthetic	`diabetes`	`california`
IRLS	warm	$1.8 \cdot 10^{-8}$ (940)	$<10^{-11}$ (894)	$<10^{-11}$ (102)
IRLS	cold	$1.8 \cdot 10^{-8}$ (922)	$<10^{-11}$ (1 118)	$<10^{-11}$ (115)
SGPTL	warm	$4.8 \cdot 10^{-5}$	2.62	0.46
SGPTL	cold	$2.4 \cdot 10^{-5}$	1.74	$\sim$64

Table 5.1: Final gap $|f(\mathbf{w}_{\text{final}}) - f^*|$ on each instance; the parenthesised number after each IRLS entry is the iteration count at which $\varepsilon_{\text{stop}} = 10^{-12}$ triggers. IRLS reaches f^* to working precision on all three instances from both starts (limited by the $\varepsilon_{\text{thr}} = 10^{-8}$ smoothing floor on the synthetic, by $\varepsilon_{\text{stop}}$ on the real data); the warm vs cold contrast is therefore in iteration count, not final accuracy. SGPTL runs 8 000 iterations everywhere with the defaults of Section 5.4; here the comparison is in the final gap. The f^* reference is the sklearn-converged value on the synthetic and the IRLS-converged value on the real data, cross-validated against CVXPY-Clarabel to ten significant digits (Section 5.6); the IRLS gaps on real data are therefore measured against IRLS itself and bound only the residual movement past the stopping criterion, not the absolute distance to the Clarabel-verified f^* (which is at most $7 \cdot 10^{-8}$ on `california`, $4 \cdot 10^{-9}$ on `diabetes`).

Order-of-magnitude check on the SGPTL real-data gaps. The theorem reads $\bar{f}^k - f^* \le L\,\|\mathbf{w}_0 - \mathbf{w}^*\| / (\gamma_{\min}\sqrt{k+1})$. Collapsing the numerator $L\,\|\mathbf{w}_0 - \mathbf{w}^*\|$ into the initial objective gap $g_0 = f(\mathbf{w}_0) - f^*$ (same scaling) gives the rule of thumb $\bar{f}^k - f^* \lesssim g_0/(\gamma_{\min}\sqrt{k})$. With cold start and standardised targets, $g_0 \approx M_{\text{train}}/2$ and $\gamma_{\min} = 0.05$, so at $k = 8\,000$ the envelope predicts $\sim$5.6 on the synthetic, $\sim$28 on `diabetes`, $\sim$1 320 on `california`. The observed gaps ($2.4 \cdot 10^{-5}$, 1.74, $\sim$64) sit one to five orders of magnitude below it. The envelope is loose because the worst-case L and sublevel-set radius overestimate the actual quantities entering the bound on a well-conditioned design with normalised columns, and the deflection floor itself binds only on degenerate steps (Section 3.2). On the synthetic the margin is widest; on `california` it shrinks because the sublevel-set radius is largest there.

IRLS. Both starts converge to f^* on every instance; the IRLS curves in Figure 5.4 flatten to the $\varepsilon_{\text{stop}}/\varepsilon_{\text{thr}}$ floor regardless of start, so the comparison is in

iteration count, not final accuracy: cold needs ∼25% more iterations than warm on `diabetes` (1 118 vs 894) and ∼13% more on `california` (115 vs 102); on the synthetic the two coincide within rounding. The pattern matches the linear rate of Theorem 2.2, in which $\|\mathbf{w}_0 - \mathbf{w}^*\|$ enters multiplicatively.

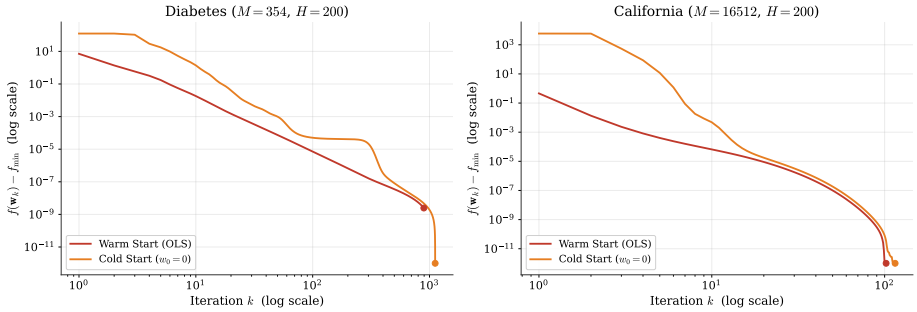

Figure 5.4: IRLS warm (OLS) vs cold ($\mathbf{w}_0 = \mathbf{0}$) start on the two ELM-transformed real datasets ($H = 200$, $\lambda_{\text{LASSO}} = 0.1$, $k_{\max} = 2000$, $\varepsilon_{\text{stop}} = 10^{-12}$), log–log axes. Both starts converge to f^* on each instance, as the linear rate of Theorem 2.2 allows; the warm curve sits below the cold one throughout because $\|\mathbf{w}_0 - \mathbf{w}^*\|$ enters the bound multiplicatively. The synthetic analogue is in Appendix B.

SGPTL. The sublinear rate makes the warm–cold contrast much less pronounced than for IRLS: the bound (3.7) shrinks the residual as $1/\sqrt{i}$, so unless $\mathbf{w}_{\text{OLS}}$ is already very close to $\mathbf{w}^*$ the two starts end the budget at comparable gaps. On the synthetic instance they finish within a factor of two (Figure 5.5: cold $2.4 \cdot 10^{-5}$ with 26 δ-contractions, warm $4.8 \cdot 10^{-5}$ with 22). On `diabetes`-ELM the cold trajectory dips below the warm one past $\sim 10^2$ iterations and stays there (1.74 vs 2.62; contraction counts 23 vs 25, so the difference is not driven by the patience schedule).

`California`-ELM is the qualitative outlier and the source of the warm-start caveat referenced throughout this report. $f(\mathbf{w}_{\text{OLS}})$ sits within 0.46 of f^* before SGPTL takes its first step, and the Polyak stepsize $\alpha_i = \beta_i(f(\mathbf{w}_i) - \text{target})/\|\mathbf{d}_i\|_2^2$ shrinks with the current gap, so the warm-started iterate barely moves over 8 000 iterations; the warm-start number 0.46 measures how close OLS already is to f^* on this instance, not the residual SGPTL reduced from it. The cold start, starting $\sim 5\,900$ above f^*, closes two decades of gap (down to ∼64) over the same budget. Figure 5.12 (Section 5.6) overlays both SGPTL starts.

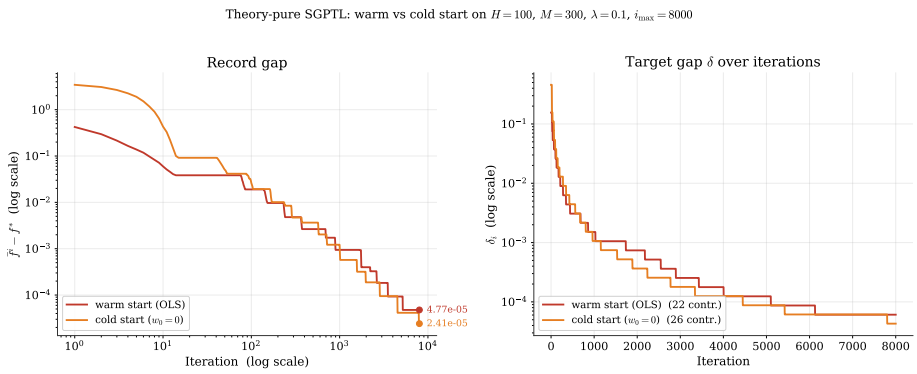

Figure 5.5: SGPTL warm vs cold start on the synthetic instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $R = 1$). Left: record gap, log–log; both descend sublinearly. Right: δ_i staircase (26 contractions cold, 22 warm).

Defaults. For IRLS we use the OLS warm start everywhere in this report. The OLS factorisation costs $O(MH^2 + H^3)$ in floating-point operations and, measured directly, 0.13 ms on `diabetes` ($H = 200$) and 2.9 ms on `california` ($H = 200$), against an IRLS body of 165 ms and 1577 ms respectively (`warm_start_cost.csv`); the warm-start share is below 0.2% of the total in both regimes, so the $\sim$13–25% iteration savings reported above more than offset it. For SGPTL we adopt the cold start $\mathbf{w}_0 = \mathbf{0}$ as the default (it is the method's natural initialisation) and pass an explicit warm start only where its final gap is empirically smaller, as on `california` in Table 5.8.

5.3.1 SGPTL beyond the submitted iteration budget

The estimate above predicts the SGPTL real-data gaps at $k = 8\,000$ from the rule of thumb $g_0/\sqrt{k}$. We extend the run to characterise the asymptotic descent of the gap: SGPTL cold to $i_{\max} = 10^7$ on `diabetes`-ELM (3.3 min wall-clock) and to $i_{\max} = 10^6$ on `california`-ELM (23 min). Figure 5.6 overlays the empirical record gap against the heuristic envelope $g_0/\sqrt{i}$.

The empirical trace stays *below* the envelope and descends faster than its worst-case slope (Table 5.2). On `diabetes` the gap drops from 0.26 at $k = 8\,000$ to $\leq 10^{-12}$ by $k = 8{\cdot}10^6$: SGPTL reaches the floating-point floor on this instance within the extended budget. On `california` it descends from 41 at $k = 8\,000$ to $1.2 \cdot 10^{-6}$ at $k = 8 \cdot 10^5$.

	`diabetes`-ELM ($g_0 = 124$)		`california`-ELM ($g_0 = 5\,898$)	
k	observed	$g_0/\sqrt{k}$	observed	$g_0/\sqrt{k}$
8 000	$2.60 \cdot 10^{-1}$	1.39	$4.14 \cdot 10^{1}$	$6.59 \cdot 10^{1}$
80 000	$2.27 \cdot 10^{-5}$	$4.39 \cdot 10^{-1}$	$7.50 \cdot 10^{-1}$	$2.09 \cdot 10^{1}$
800 000	$3.40 \cdot 10^{-7}$	$1.39 \cdot 10^{-1}$	$1.20 \cdot 10^{-6}$	6.59
8 000 000	$\leq 10^{-12}$	$4.39 \cdot 10^{-2}$	—	—
10 000 000	$\leq 10^{-12}$	$3.92 \cdot 10^{-2}$	—	—

Table 5.2: Observed record gap of SGPTL versus the $g_0/\sqrt{k}$ envelope, cold start on the ELM-transformed real datasets.

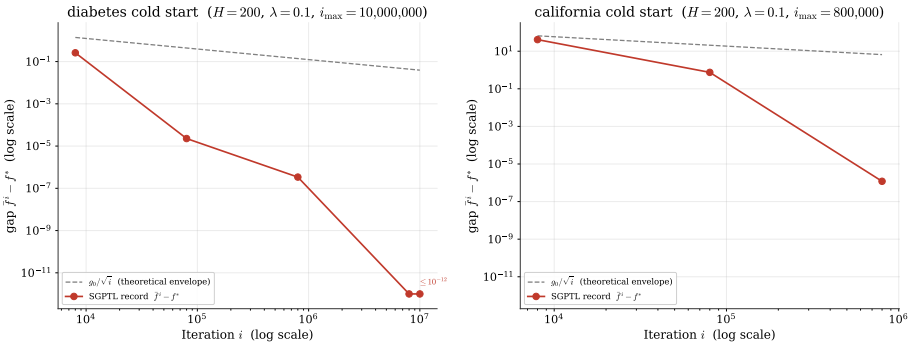

Figure 5.6: SGPTL at large iteration budgets, cold start on the ELM-transformed real datasets ($H = 200$, $\lambda_{\text{LASSO}} = 0.1$), log–log. Empirical record gap (solid) vs the heuristic envelope $g_0/\sqrt{i}$ (dashed grey). Left: `diabetes` to $i_{\max} = 10^7$, reaches the 10^{-12} floor by $\sim 8 \cdot 10^6$ iterations. Right: `california` to $i_{\max} = 8 \cdot 10^5$, gap $\sim 10^{-6}$ at the budget.

5.4 Hyperparameter tuning and technique choices

This section reports sensitivity sweeps on each algorithm separately: the IRLS parameters ε_{thr} and λ_{LASSO} and the choice of inner solver (Cholesky vs Jacobi-preconditioned CG) below, the SGPTL parameters $\gamma_{\min}$, δ_0 and ρ after.

5.4.1 IRLS: ε_{thr} and λ_{LASSO}

We sweep both at $H = 100$, $M = 400$, 100 iterations (Figure 5.7). For $\varepsilon_{\text{thr}} \in \{10^{-4}, \ldots, 10^{-12}\}$ the final gap tracks ε_{thr} ($1.4 \cdot 10^{-6}$, $1.5 \cdot 10^{-8}$, $5.3 \cdot 10^{-10}$ at $10^{-6}, 10^{-8}, 10^{-10}$); 10^{-4} is too loose (no sparsity) and 10^{-12} hits floating-point limits, so the 10^{-8}–10^{-10} range is the default. Sweeping $\lambda_{\text{LASSO}} \in \{0.01, \ldots, 1.0\}$ leaves the convergence speed roughly invariant (λ enters $\mathbf{Q}_k$ as a bounded conditioning factor) while sparsity moves monotonically from 11% to 95%, as the optimality condition (1.5) implies.

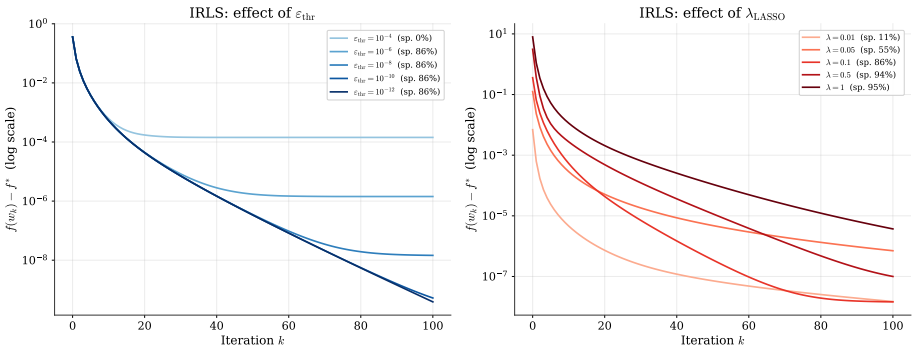

Figure 5.7: IRLS sensitivity. Left: ε_{thr} at $\lambda_{\text{LASSO}} = 0.1$. Right: λ_{LASSO} at $\varepsilon_{\text{thr}} = 10^{-8}$.

5.4.2 IRLS: inner solver (Cholesky vs. CG)

Chapter 2 leaves the inner solver of $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$ open between Cholesky ($O(H^3/3)$) and CG ($O(pH^2)$ per step). Table 5.3 compares them on $H = 100$, $M = 400$: both reach the stopping criterion at iteration 265 with identical trajectories, but exact Cholesky takes 11.7 ms against 24.4 ms for Jacobi-preconditioned CG. As IRLS drives inactive weights to the cap $1/\varepsilon_{\text{thr}} = 10^8$, $\kappa(\mathbf{Q}_k)$ blows up; the Jacobi preconditioner on $\text{diag}(\mathbf{Q}_k)$ is what keeps CG matching Cholesky iteration-by-iteration, but the iterative overhead still doubles the wall-clock time. In the project regime ($M \gg H$, $H \leq 1000$) Cholesky is the default: it is faster and the exact solve preserves MM-descent unconditionally with no inner tolerance to tune. CG becomes an alternative for H large enough that the $O(H^3)$ term dominates.

Solver	Total time (ms)	Iterations	Sparsity at 10^{-6}	Converged
Cholesky	11.7	265	86%	Yes
CG	24.4	265	86%	Yes

Table 5.3: IRLS solver comparison on $H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, $\varepsilon_{\text{thr}} = 10^{-8}$, $k_{\max} = 300$, OLS warm start. Both reach $\varepsilon_{\text{stop}} = 10^{-10}$ at iteration 265; sparsity at the final iterate under $|w_i| < 10^{-6}$.

5.4.3 SGPTL: the deflection floor $\gamma_{\min}$

Theorem 3.1 requires $\gamma_i \geq \gamma_{\min} > 0$ to keep the stepsize clip $\beta_i = \min(\beta, \gamma_i)$ uniformly bounded away from zero (Section 3.2); the greedy formula (3.4) of Section 3.2.1 can otherwise drive γ_i arbitrarily close to zero near stationarity. We run two sweeps below to (i) check whether the floor is needed in practice on ELM LASSO and (ii) calibrate its value.

Does the floor matter? (binary test, $\gamma_{\min} \in \{0, 0.05\}$). We compare the algorithm with no floor ($\gamma_{\min} = 0$, the raw greedy) against the floored variant ($\gamma_{\min} = 0.05$) on a 3×5 grid: three sizes (H, M) equal to $(50, 150)$, $(100, 300)$ and $(200, 600)$, five seeds each. The same pattern emerges across the 30 runs (Figure 5.8; aggregate over the medium-size block):

$\gamma_{\min}$	final gap	$\Pr\{\gamma_i \le 0.01\}$	Pr{skip fires}	δ-contr.
0.00	$4.4 \cdot 10^{-2}$	99%	98%	~ 2
0.05	$1.2 \cdot 10^{-4}$	0%	0%	~ 19

The mechanism behind the failure mode of $\gamma_{\min} = 0$ is a chain reaction visible on the figure's leftmost panel. Near stationarity the greedy γ^* from (3.4) approaches zero because consecutive subgradients align and the numerator $\|\mathbf{d}_{i-1}\|^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle$ vanishes. With no floor, γ_i collapses below 10^{-10} from roughly iteration 50 onward and stays there for the rest of the run. The clip $\beta_i = \min(1, \gamma_i)$ collapses with it, the Polyak step (3.2) becomes positive but minuscule, and once f_i no longer beats the target the numerator-skip safeguard fires on 98% of iterations: the iterate freezes and δ contracts only ~ 2 times instead of ~ 19, leaving the gap stuck at $\sim 4 \cdot 10^{-2}$ (centre and right panels). With $\gamma_{\min} = 0.05$ the floor binds whenever the greedy would hit zero, β_i stays ≥ 0.05, the Polyak step remains effective, and the staircase descent of δ that the theory predicts takes place; the gap drops by two more orders of magnitude. This failure depends on the algorithm's greedy γ choice rather than on the specific instance, and we observed it on all 15 unfloored seeds.

Which value of $\gamma_{\min}$ (calibration sweep). We sweep $\gamma_{\min}$ over 0.05, 0.1, 0.2 and 0.5 on the convergence instance. The final gap lies in $[10^{-7}, 10^{-5}]$ across the four values, with $\gamma_{\min} = 0.05$ marginally best. Larger values force the clip to bind on iterations where the greedy γ_i would otherwise lie strictly inside $(\gamma_{\min}, 1]$, so we adopt $\gamma_{\min} = 0.05$.

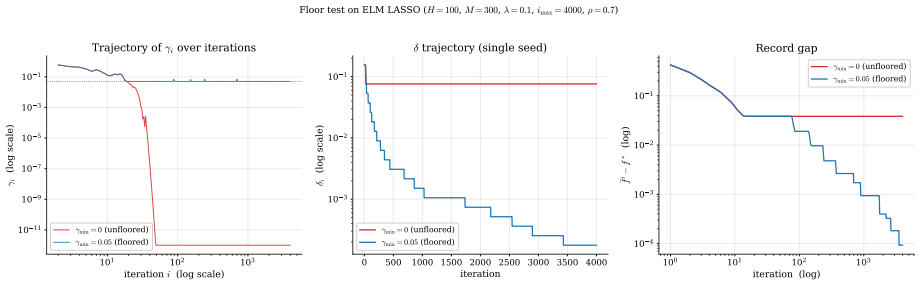

Figure 5.8: Floor test on ELM LASSO ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 4000$, $\rho = 0.7$), single seed shown. Left: trajectory of γ_i over the run (log–log). Without the floor, γ_i collapses below 10^{-10} around iteration 50 and stays there; with $\gamma_{\min} = 0.05$ it sits on the floor (dotted grey line) for the rest of the run. Centre: target gap δ_i; the unfloored δ contracts twice and stops, the floored one descends in the staircase the theory expects. Right: record gap $\bar{f}^i - f^*$; the floor buys two orders of magnitude. The aggregate over the 30 runs of the binary grid agrees with this single-seed snapshot.

5.4.4 SGPTL: δ_0 and ρ

Of the four target-level parameters of SGPTL, β and R are settled analytically in Section 3.4.1 and not swept here. The remaining two, $\delta_0 = c\, f(\mathbf{w}_0)$ and ρ, have empirical tradeoffs and are calibrated below. The sweeps of Figure 5.9 ($H = 100$, $M = 400$, 5000 iterations) show the method is robust to both: $c \in \{0.01, \ldots, 1.0\}$ at $\rho = 0.7$ gives final gaps in $[5.6 \cdot 10^{-5}, 1.6 \cdot 10^{-4}]$, within one order of magnitude across two decades of δ_0; $\rho \in \{0.3, \ldots, 0.95\}$ at $c = 0.1$ gives $1.98 \cdot 10^{-5}$ up to $8.12 \cdot 10^{-4}$ with 7 to 91 contractions. Small ρ wins on this instance, but a smoke test over the synthetic and the two ELM matrices places the per-instance best at $\rho = 0.3, 0.3, 0.7$; only $\rho \in [0.3, 0.7]$ is within a factor of two of the best everywhere, and $\rho \geq 0.9$ loses by one to two orders of magnitude. We adopt $\rho = 0.7$ and $c = 0.1$ as robust defaults.

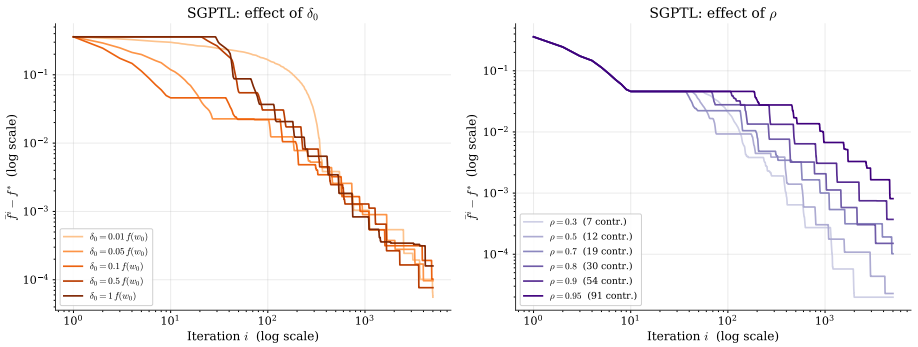

Figure 5.9: SGPTL sensitivity ($H = 100$, $M = 400$, OLS warm start, 5000 iterations). Left: δ_0 as a fraction of $f(\mathbf{w}_0)$ at $\rho = 0.7$. Right: ρ at $\delta_0 = 0.1\, f(\mathbf{w}_0)$; gaps and contraction counts grow monotonically with ρ.

5.4.5 SGPTL: δ_0 against the f^*-based scale (abstract sensitivity check)

The operational choice is $\delta_0 = c\, f(\mathbf{w}_0)$, settled in Section 3.4.1 and used in every run of this chapter. The comparison below is purely an abstract sensitivity study: we ask, as a sanity check, how the admissible scale would compare against the hypothetical $\delta_0 = c\, f^*$ *if* f^* were known. Tuning algorithmic parameters against f^* is not allowed in practice (it would amount to using the optimum to set the optimizer, the optimisation analogue of training on the test set), so the f^*-based scale enters only as a reference point in this and the next subsection and is never used to choose the default.

We run both scales on five instances over $c \in \{0.01, \ldots, 1\}$. The admissible scale stays within a factor of two of the oracle in every regime where SGPTL is in fact optimising (Table 5.4). The one exception is `diabetes` $H = 50$ at $c \geq 0.5$, where an over-aggressive δ_0 overshoots the sufficient-descent test from the first iteration and both scales stall at a common plateau (a regime limit of the sweep, not a finding about the scales). The full per-instance grid is in Appendix A.

instance	admissible / oracle gap ratio
synthetic (5 seeds)	[0.50, 1.07]
`diabetes` $H = 50$ ($c \leq 0.1$)	[0.73, 1.96]
`diabetes` $H = 200$	[0.87, 1.04]
`california` $H = 50$ (cold)	[0.72, 1.41]
`california` $H = 200$ (cold)	[0.96, 1.07]

Table 5.4: Ratio of the admissible scale $\delta_0 = c\, f(\mathbf{w}_0)$ to the inadmissible oracle $\delta_0 = c\, f^*$ on the final gap, over $c \in \{0.01, \ldots, 1\}$ in the working regime. All ratios lie in $[0.5, 2]$. Full grid: Appendix A.

5.5 IRLS versus SGPTL

Chapter 4 contrasts the linear cubic-per-iteration IRLS with the sublinear cheap-per-iteration SGPTL and predicts IRLS dominance for moderate-to-high accuracy in the $M \gg H$ regime. The following subsections test this prediction on solution quality and sparsity, scaling, and iterations to a target.

5.5.1 Solution quality and sparsity

On $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$ (reference sparsity 88%), IRLS reaches $f - f^* \leq 10^{-6}$ in 101 iterations at $\|\mathbf{w}_{\text{IRLS}} - \mathbf{w}^*\|_2 = 2.95 \cdot 10^{-6}$; SGPTL reaches $1.0 \cdot 10^{-4}$, $\sim 35\times$ looser. The two need *different* sparsity thresholds, as Section 3.6 predicts: IRLS pins inactive weights near $\varepsilon_{\text{thr}} = 10^{-8}$, whereas SGPTL's inactive components sit at the residual-rate scale $L/(\gamma_{\min}\sqrt{i}) \in [10^{-4}, 10^{-3}]$. Applying the *same* hard threshold to both (Table 5.5): at 10^{-6} SGPTL's F_1 collapses to 0.36 because its inactive coefficients sit above the cutoff; at 10^{-3}, above the residual floor, both recover the support exactly.

threshold	method	sparsity	precision	recall	F1
10^{-6}	IRLS	86%	0.857	1.000	0.923
10^{-6}	SGPTL	46%	0.222	1.000	0.364
10^{-6}	ground truth	88%	1.000	1.000	1.000
10^{-3}	IRLS	88%	1.000	1.000	1.000
10^{-3}	SGPTL	88%	1.000	1.000	1.000
10^{-3}	ground truth	88%	1.000	1.000	1.000

Table 5.5: Support recovery on $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$ (reference sparsity 88%): same hard threshold on both methods, precision/recall/F1 against the support of $\mathbf{w}^*$. At 10^{-3} both recover it exactly; at 10^{-6} SGPTL fails because its inactive coefficients sit at the $L/\sqrt{i} \sim 10^{-3}$ scale.

5.5.2 Scalability

Setting. We grow the ELM size $H \in \{50, 100, 500, 1000, 2000\}$ at $M = 5H$ on the synthetic instance and time both algorithms at a *fixed iteration budget*: 100 Cholesky iterations for IRLS, 3000 subgradient iterations for SGPTL. The per-iteration costs $O(MH^2 + H^3)$ for IRLS and $O(MH)$ for SGPTL are derived in Chapter 4 (Table 4.1) and appear as the $O(H^3)$ and $O(H^2)$ slopes drawn in Figure 5.10 (left) at $M = 5H$.

What we measure. Both algorithms scale up with H (Table 5.6), but IRLS stays faster than SGPTL throughout the tested range and reaches a final gap roughly two orders of magnitude smaller. At $H = 2000$: IRLS takes 2.4 s to reach gap 10^{-5}; SGPTL takes 30 s to reach $5.3 \cdot 10^{-4}$. The right panel of

Figure 5.10 confirms the per-iteration ordering (SGPTL is cheaper there), but the cumulative time inverts because SGPTL needs many more iterations.

Why the fixed-budget table flatters SGPTL. This comparison fixes iterations, not accuracy. To reach a target ε Theorem 3.1 forces $\Omega(L^2/(\gamma_{\min}^2\varepsilon^2))$ SGPTL iterations with L growing in H; the cost-to-accuracy ratio is then much wider than Table 5.6 suggests, as quantified in Section 5.5.3.

H	M	t_{IRLS} (s)	gap_{IRLS}	t_{SGPTL} (s)	$\text{gap}_{\text{SGPTL}}$
50	250	0.003	$5.1 \cdot 10^{-7}$	0.060	$9.6 \cdot 10^{-5}$
100	500	0.004	$1.8 \cdot 10^{-7}$	0.079	$1.0 \cdot 10^{-4}$
500	2500	0.126	$1.8 \cdot 10^{-6}$	0.514	$3.2 \cdot 10^{-4}$
1000	5000	0.624	$7.6 \cdot 10^{-6}$	10.54	$4.7 \cdot 10^{-4}$
2000	10000	2.44	$1.0 \cdot 10^{-5}$	29.62	$5.3 \cdot 10^{-4}$

Table 5.6: Total wall-clock time and final gap as H grows at $M = 5H$. IRLS: 100 Cholesky iterations; SGPTL: 3000 subgradient iterations.

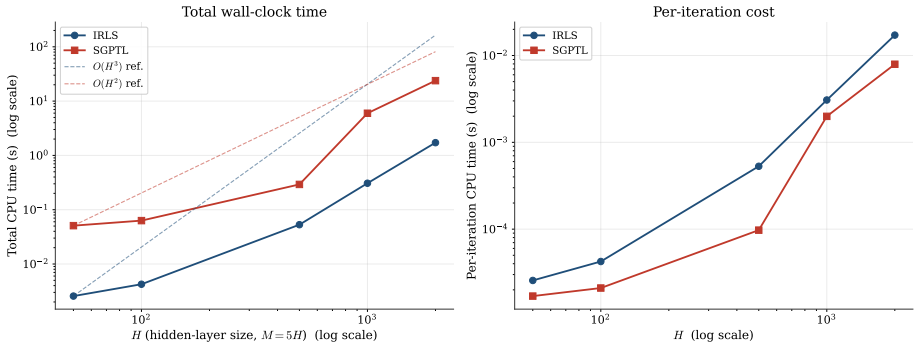

Figure 5.10: Scaling with H at $M = 5H$. Left: total time, log–log, with $O(H^3)$ (IRLS) and $O(H^2)$ (SGPTL) references anchored on the smallest-H point. Right: per-iteration cost.

5.5.3 Iterations to a target accuracy

Table 5.7 and Figure 5.11 report the cost to reach $f - f^* \le \varepsilon$ on the moderate problem ($H = 50$, $M = 200$). At the loosest accuracy the two are competitive (2 vs 1 at 10^{-1}); past that the gap widens fast: SGPTL needs $108, 674, 2477$ iterations for $10^{-2}, 10^{-3}, 10^{-4}$ against IRLS's $3, 7, 19$ (54 ms vs 0.7 ms at 10^{-4}). The per-decade ratio predicted by Chapter 4 (constant increment for IRLS, $\sim$100$\times$ multiplier for SGPTL) is reproduced here; SGPTL does not reach 10^{-6} within 10 000 iterations, IRLS reaches it in 101.

ε	k_{IRLS}	t_{IRLS} (s)	i_{SGPTL}	t_{SGPTL} (s)
10^{-1}	1	$7.8 \cdot 10^{-5}$	2	$1.2 \cdot 10^{-4}$
10^{-2}	3	$1.6 \cdot 10^{-4}$	108	$3.2 \cdot 10^{-3}$
10^{-3}	7	$3.3 \cdot 10^{-4}$	674	$1.5 \cdot 10^{-2}$
10^{-4}	19	$6.9 \cdot 10^{-4}$	2477	$5.4 \cdot 10^{-2}$
10^{-6}	101	$8.1 \cdot 10^{-3}$	—	—

Table 5.7: Iterations and time to reach $f - f^* \leq \varepsilon$ on $H = 50$, $M = 200$, $\lambda_{\mathrm{LASSO}} = 0.1$.

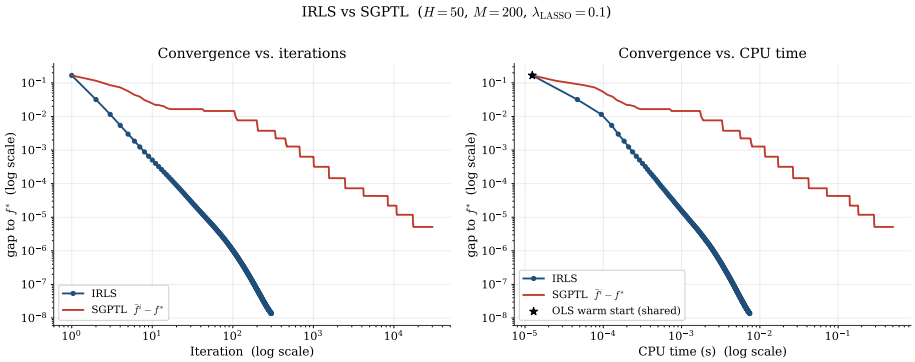

Figure 5.11: IRLS vs SGPTL on $H = 50$, $M = 200$, $\lambda_{\mathrm{LASSO}} = 0.1$, log–log. Left: gap vs iteration. Right: gap vs CPU time. SGPTL falls below 10^{-2} in 108 iterations and 10^{-3} in 674 then enters the sublinear regime; IRLS reaches 10^{-6} in 101 iterations ($\sim$8 ms).

5.6 Validation on real datasets

We need an independent f^* to report gaps on these two instances. The natural off-the-shelf candidate is sklearn's coordinate-descent `Lasso`, but on the ELM-transformed matrices sklearn never reaches its own `tol` stop: at `max_iter` $= 10^5$, `tol` $= 10^{-10}$, it returns at the iteration cap with a final duality gap two to six orders of magnitude above tolerance, and a residual gap to f^* of $+1.4 \cdot 10^{-3}$ on `diabetes` and $+2.82$ on `california`. sklearn CD is therefore not a usable oracle on these problems, and we build f^* instead by running IRLS to convergence ($k_{\max} = 3000, \varepsilon_{\mathrm{thr}} = 10^{-14}$) and cross-validating it against CVXPY's interior-point solver Clarabel. The two are structurally different (MM + linear solve vs. primal–dual interior point) and agree to ten significant digits on both instances.

Both datasets use $H = 200$ sigmoid units, $\lambda_{\mathrm{LASSO}} = 0.1$. The comparison criterion is fixed-accuracy rather than fixed-iteration: each method runs until $f - f^* \leq 10^{-6}$, and the cost is reported in iterations and wall time (Table 5.8). IRLS runs with tight smoothing $\varepsilon_{\mathrm{thr}} = 10^{-12}$ from the OLS warm start;

CVXPY-Clarabel runs at `tol_gap`$= 10^{-9}$; SGPTL runs cold from $\mathbf{w}_0 = \mathbf{0}$ with $\beta = 1$, $\gamma_{\min} = 0.05$, $R = 1$, $\rho = 0.7$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$. The SGPTL iter-to-target counts and wall-times in Table 5.8 are measured directly: we rerun SGPTL with $i_{\max}$ set just above the long-run crossing ($3.6 \cdot 10^5$ on `diabetes`, $1.03 \cdot 10^6$ on `california`) and read the wall-clock at the first iteration where the record gap falls below 10^{-6} (`real_data_crossing.csv`). The full long-run trajectory to $i_{\max} = 10^7/10^6$ is the one of Section 5.3.1.

method	iter to gap$\leq 10^{-6}$	wall time	sp. 10^{-3}	test MSE
Diabetes ($M = 354$, $f^* = 52.948763$)				
CVXPY-Clarabel	10	92 ms	19%	0.898
IRLS	178	14.5 ms	19%	0.898
SGPTL (cold)	$3.24 \cdot 10^5$	7.7 s	19%	0.898
Ridge	1 (closed form)	< 1 ms	—	0.942
California ($M = 16512$, $f^* = 2358.019449$)				
CVXPY-Clarabel	8	6.3 s	4%	0.307
IRLS	34	18.7 ms	4%	0.307
SGPTL (cold)	$9.29 \cdot 10^5$	1082 s (18.0 min)	4%	0.307
Ridge	1 (closed form)	< 5 ms	—	0.307

Table 5.8: Cost to reach a uniform target accuracy $f - f^* \leq 10^{-6}$ on the two ELM-transformed real datasets, $H = 200$, $\lambda_{\text{LASSO}} = 0.1$, single-thread CPU timings. The reference f^* is the cross-validated value of the introduction (IRLS at $\varepsilon_{\text{thr}} = 10^{-14}$, $k_{\max} = 3000$ against CVXPY-Clarabel at `tol_gap`$= 10^{-9}$). The *iter* column counts the inner loop of each method (Newton interior-point steps for Clarabel, IRLS outer iterations each one Cholesky solve, SGPTL subgradient steps); Ridge requires one Cholesky solve and is given as a sparsity-less baseline. Wall times measure the iteration loop only; the one-time OLS warm-start factorisation (0.13 ms on `diabetes`, 2.9 ms on `california`, Section 5.3) is excluded, as is the analogous one-time $\delta_0 = c\, f(\mathbf{w}_0)$ evaluation paid by SGPTL ($O(M)$, microseconds). Sparsity counts $|w_i| < 10^{-3}$ (Section 5.5.1); test MSE is on the held-out 20% split and is reported for completeness only (cf. "reading the table" paragraph).

Reading the table. At the same target accuracy the three optimisers agree on the solution (same sparsity, all distinct from Ridge on `diabetes`) but disagree on cost by orders of magnitude. The test-MSE columns are included for completeness and are not part of the optimisation comparison: out-of-sample error is a learning metric, while the question here is how cheaply each algorithm reaches the same training optimum. IRLS is fastest in wall time on both datasets (14.5 ms, 18.7 ms): the rate is linear in the active-set regime, and the linear cost per iteration ($O(MH^2 + H^3)$) is paid only a few dozen times. Clarabel needs the fewest iterations (8–10) but each one factorises an $H \times H$ system, so on `california` ($M = 16512$) its wall time is $\approx$340$\times$ that of IRLS; the itera-

tion ranking and the wall-time ranking diverge whenever the per-iteration cost differs sharply across methods. SGPTL pays the sublinear rate: $3.3 \cdot 10^5$ iterations on `diabetes` and $9.3 \cdot 10^5$ on `california` to reach the same 10^{-6} target, $\sim 10^3$ to 10^4 times the iteration count of the other two and a wall-time loss of $\sim 10^2$–$10^5\times$. Figure 5.12 gives the per-iteration view: IRLS hits its accuracy floor at ~ 100 iterations, SGPTL follows the $g_0/\sqrt{i}$ envelope of Theorem 3.1 for 10^5–10^7 iterations.

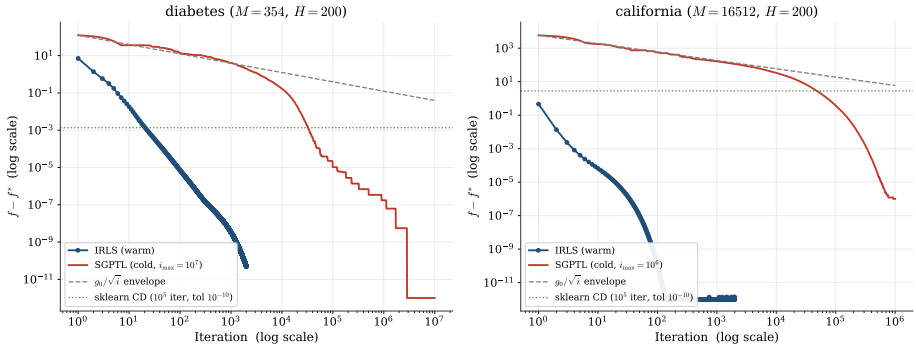

Figure 5.12: Convergence on the two real datasets, log–log, $f - f^*$ against iteration. IRLS (blue, $k_{\max} = 2000$, $\varepsilon_{\text{thr}} = 10^{-12}$) reaches the f^* floor in ~ 100 iterations on both. SGPTL cold (red, long-run trace from Section 5.3.1: $i_{\max} = 10^7$ on `diabetes`, 10^6 on `california`) follows the $g_0/\sqrt{i}$ envelope predicted by Theorem 3.1 (dashed grey) and reaches gap $\leq 10^{-12}$ on `diabetes` and $\approx 10^{-6}$ on `california`. The horizontal green dotted line marks where sklearn CD returns at `max_iter` $= 10^5$ (included to make the iteration-budget gap visible: SGPTL closes that residual at $\sim 10^6$ extra iterations, sklearn cannot reach 10^{-6} on either dataset within its own tolerance).

5.7 Initial design and corrections

Two SGPTL design choices in Algorithm 2 were reached only after correcting an earlier implementation. We document each as *before / why wrong / after / why right*, then compare the numbers the two implementations produce on the convergence instance.

Target-level scale δ_0. *Before:* $\delta_0 = c\, f^*$. *Why wrong:* f^* is the very quantity SGPTL exists to estimate, so using it to set an algorithmic parameter assumes access to the answer and disqualifies the result. *After:* $\delta_0 = c\, f(\mathbf{w}_0)$. *Why right:* $f(\mathbf{w}_0)$ is computable from the starting point alone; the sweep in Section 5.4.5 confirms it tracks the oracle scale within a factor of two on every test instance, so the admissible choice loses nothing in practice.

Contraction trigger and the radius R. *Before:* $R = 10\sqrt{i_{\max}} \approx 894$ together with an iteration-count fallback that contracted δ every $\lceil i_{\max}/30 \rceil$ steps. *Why wrong:* the realistic per-iteration step length on this problem is $\alpha_i \|\mathbf{d}_i\| \sim 10^{-3}$, so cumulative travel never reaches $R \approx 894$ and the travel-distance branch never fires; *every* contraction is driven by the iteration counter, which is inconsistent with [7, Lemma 3.8]'s requirement that each contraction follow $r_k > R$ travel. *After:* $R = 1$ matched to the realistic step length, fallback removed. *Why right:* contractions now fire on travel as the lemma requires, so the δ schedule reflects the actual descent.

Before vs. after on the convergence instance. Table 5.9 compares the two implementations on the same instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$). The pre-correction contraction count is $\sim$95 (all from the fallback, none from travel-distance); post-correction it is $\sim$22 (all from travel-distance). The record gap drops by three orders of magnitude under the old setup, but this is not convergence: it is the δ schedule collapsing on a counter rather than on the optimisation state, decoupling target levels from actual progress. The post-correction record gap $4.8 \cdot 10^{-5}$ is the figure the theory of Theorem 3.1 bounds for SGPTL on this instance. The qualitative ranking against IRLS (Chapter 4) is unaffected.

	before (wrong)	after (correct)
δ_0	$c\,f^*$ (oracle)	$c\,f(\mathbf{w}_0)$
R used	$10\sqrt{i_{\max}} \approx 894$	1
iteration-count fallback	yes	no
contractions from $r_k > R$	0	$\sim$22
contractions from fallback	$\sim$95	0
final record gap $\bar{f} - f^*$	$\sim 6.6 \cdot 10^{-8}$	$4.8 \cdot 10^{-5}$

Table 5.9: Pre- and post-correction SGPTL numbers on the convergence instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$). The pre-correction record gap is lower in absolute terms but is produced by a δ schedule that does not track descent, so its meaning as a measure of optimisation progress is not what the theorem bounds.

Chapter 6

Conclusions

We studied the ELM LASSO problem of Section 1.3 with two algorithms at opposite ends of the nonsmooth-optimisation spectrum: IRLS (Chapter 2), which reformulates the ℓ_1 penalty as a sequence of smooth quadratic surrogates and inherits the MM linear rate of Theorem 2.2, and SGPTL (Chapter 3), which keeps the non-smoothness explicit and pays the $\Omega(L^2/\varepsilon^2)$ first-order lower bound of Theorem 3.1. Chapter 4 combined the two rates with the per-iteration cost analysis (Table 4.1) to predict IRLS dominance whenever a moderate-to-high accuracy is wanted in the $M \gg H$ regime, and the experiments of Chapter 5 test that prediction.

Experimental confirmation. The predictions of Chapter 4 are reproduced on every axis tested: linear vs sublinear rate on the synthetic convergence instance (Section 5.2); fixed-accuracy cost separation of orders of magnitude on the ELM-transformed real datasets (Section 5.6); per-decade iteration cost constant for IRLS and $\sim 100\times$ for SGPTL (Section 5.5.3). Clarabel converges in the fewest iterations but factorises an $H\times H$ system at each one and ends $\sim 340\times$ slower than IRLS on `california`. Sklearn's coordinate descent does not reach 10^{-6} on either ELM matrix within its own tolerance budget and is not a usable oracle in this regime. SGPTL closes the gap to f^* at large budgets on the real data (Section 5.3.1), as Theorem 3.1 predicts.

Recommendation for the ELM LASSO problem. In the intended regime ($M \gg H$, fixed $\mathbf{W}_1$, output layer trained offline), IRLS is the recommended solver on the basis of: faster wall-clock at every tested $H \leq 2000$ (Section 5.5.2); fastest to 10^{-6} on both real datasets with agreement to Clarabel within ten significant digits (Section 5.6); hard sparsity at ε_{thr} with no thresholding post-step (Section 5.5.1). SGPTL would become competitive only when $H^2 \gg M$ or when $\mathbf{X}^\top\mathbf{X}$ does not fit in memory, regimes that do not appear at the hidden-layer sizes tested.

Limitations and possible extensions. The evidence base is two ELM-transformed real datasets and one synthetic family, all with $H \leq 2000$ and $\kappa(\mathbf{X}^\top \mathbf{X})$ up to $\sim 10^6$ (Section 5.1.1). We did not test designs substantially more ill-conditioned, nor H large enough that the $O(H^3)$ IRLS factorisation dominates while the cheaper $O(MH)$ SGPTL iteration becomes competitive. The warm-vs-cold SGPTL contrast on `california` is partly a statement about the data (Section 5.3). A stochastic or block-coordinate version of SGPTL would lower the $O(MH)$ iteration cost across mini-batches or coordinate blocks; this targets the $H^2 \gg M$ or out-of-core regimes where IRLS' factorisation is the bottleneck. On the IRLS side, the inner Cholesky solve can be replaced by an inexact iterative solver with a tolerance tied to the outer MM progress, lowering the $O(H^3)$ factor at large H while preserving the linear outer rate of Theorem 2.2.

Appendix A

Full δ_0 family sweep (abstract sensitivity)

This appendix expands Section 5.4.5. The whole construction is an abstract sensitivity study: the operational default of the report is the admissible scale $\delta_0 = c\, f(\mathbf{w}_0)$, calibrated on principled grounds without reference to f^* (Section 3.4.1). The f^*-based scale below is reported only as a reference; it is not used to choose any parameter.

We compare Family A $\delta_0 = c\, f(\mathbf{w}_0)$ against the (inadmissible) Family C $\delta_0 = c\, f^*$ over $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$ on five instances. On four of the five f^* is CVXPY-verified (`CLARABEL`, cross-validated against the IRLS-converged value to relative agreement below 10^{-9}); on `california` $H = 200$ the size puts CVXPY beyond a reasonable budget and we substitute the IRLS-converged proxy — an internal double-circularity (the proxy comes from the algorithm being compared, and is then fed back as f^* to define the oracle scale) that we flag here so the row is read as illustrative, not as a validation of the calibration. The instance configurations are summarised in Table A.1.

Family B $\delta_0 = c\, \frac{1}{2} \|\mathbf{X}\mathbf{w}_0 - \mathbf{y}\|^2$ (the smooth-part-only variant of A) is plotted for completeness but is not adopted: it has no theoretical preference over A and is empirically comparable.

The `california` $H = 50$ and $H = 200$ rows are run from the cold start $\mathbf{w}_0 = \mathbf{0}$ because the OLS warm start on those instances already sits very close to f^* (Section 5.3); the warm-started run would leave SGPTL nothing to optimise and the three families would coincide at the starting gap.

instance	seeds	$M \times H$	start	f^* source
synthetic	5	300×100	OLS warm	CVXPY-verified
`diabetes` $H = 50$	1	354×50	OLS warm	CVXPY-verified
`diabetes` $H = 200$	1	354×200	OLS warm	CVXPY-verified
`california` $H = 50$	1	16512×50	cold $\mathbf{w}_0 = \mathbf{0}$	CVXPY-verified
`california` $H = 200$	1	16512×200	cold $\mathbf{w}_0 = \mathbf{0}$	IRLS-converged proxy

Table A.1: Instances used in the δ_0 family sweep. `california` runs cold because the OLS solution coincides with f^* to machine precision at $\lambda_{\text{LASSO}} = 0.1$. All runs share $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $\beta = 1$, $R = 1$.

Across every instance, in the regime where SGPTL is optimising, Family A and the oracle Family C agree within a factor of two (Table 5.4). The single exception is `diabetes` $H = 50$ at $c \geq 0.5$, where both scales stall at a common plateau $\bar{f}^i - f^* \approx 1.4 \cdot 10^{-1}$ because the over-aggressive δ_0 overshoots the sufficient-descent test from the first iteration and only the travel-distance counter drives progress; they coincide there because neither recovers from the initial overshoot, which is a regime limit of the sweep rather than a statement about the calibration.

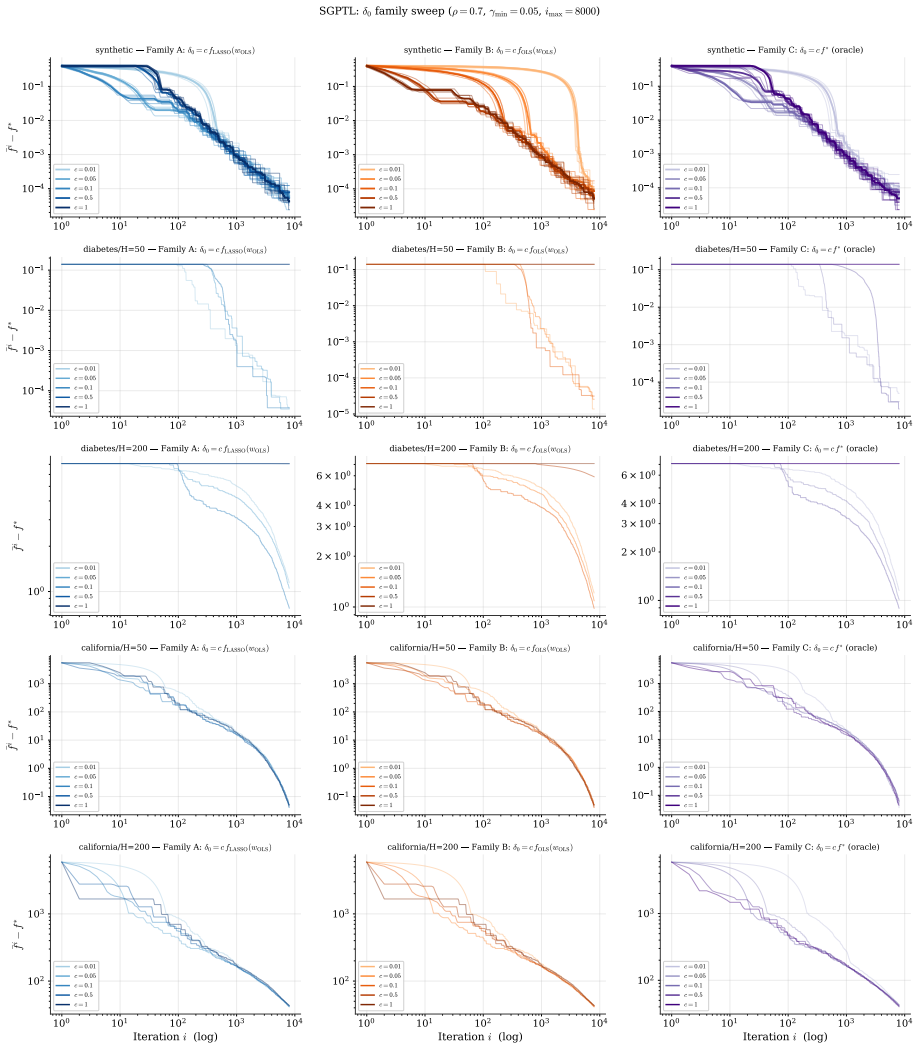

Figure A.1: Sensitivity of the final record gap to δ_0, three families across five instances (rows): synthetic (5 seeds, per-seed thin traces and aggregate-mean thick), `diabetes` $H = 50$ (warm), `diabetes` $H = 200$ (warm), `california` $H = 50$ (cold), `california` $H = 200$ (cold). All runs use $\lambda_{\mathrm{LASSO}} = 0.1$, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $i_{\max} = 8000$. Family A: $\delta_0 = c\, f(\mathbf{w}_0)$ (default). Family B: $\delta_0 = c\, \frac{1}{2} \|\mathbf{X}\mathbf{w}_0 - \mathbf{y}\|^2$. Family C: $\delta_0 = c\, f^*$ (oracle, inadmissible). The Family A/Family C ratio falls in $[0.5, 2.0]$ in every regime where SGPTL admits progress.

Appendix B

IRLS warm vs cold start on the synthetic instance

Section 5.3 reports the warm-vs-cold contrast for IRLS on the two real-data instances directly in the body (Figure 5.4). The analogue on the synthetic benchmark ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, 200 iterations, $\varepsilon_{\text{thr}} = 10^{-8}$) is reported here for completeness.

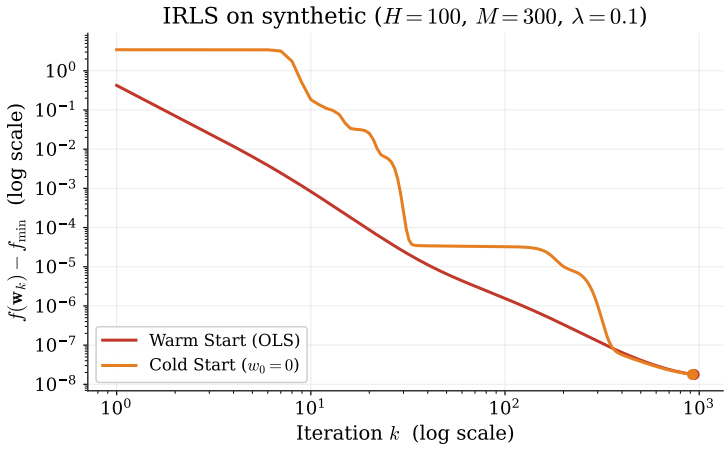

Figure B.1: IRLS warm (OLS) vs cold ($\mathbf{w}_0 = \mathbf{0}$) start on the synthetic instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $k_{\max} = 2000$, $\varepsilon_{\text{thr}} = 10^{-8}$), log–log axes. Both starts converge to the ε_{thr} floor ($\bar{f} - f^* \approx 1.8 \cdot 10^{-8}$) at iteration 940 (warm) and 922 (cold); the warm curve sits below the cold one throughout the descent, consistent with the multiplicative role of $\|\mathbf{w}_0 - \mathbf{w}^*\|$ in the bound of Theorem 2.2.

Appendix C

Derivation of the per-step Polyak inequality (3.8)

We derive the per-step descent estimate (3.8) of the proof of Theorem 3.1 from three ingredients of [7]: the basic identity (2.9), Corollary 3.2, and the Polyak stepsize (3.2). The notation map between the paper and the report is $\alpha_k^{\text{paper}} \leftrightarrow \gamma_i$, $\beta_k^{\text{paper}} \leftrightarrow \beta_i$, $\nu_k^{\text{paper}} \leftrightarrow \alpha_i$; the exact-oracle regime fixes $\sigma_k = 0$, the uncorrected Polyak variant fixes the inner correction γ_k of (3.1) of [7] to zero, and we evaluate the result at $\bar{x} = \mathbf{w}^*$.

Step 1 — Identity (2.9). In the unconstrained case $X = \mathbb{R}^H$ no projection is needed and (2.9) of [7] reduces to expanding the square $\|\mathbf{w}_i - \alpha_i \mathbf{d}_i - \mathbf{w}^*\|_2^2$:

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \;=\; \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 \;-\; 2\alpha_i \,\langle \mathbf{d}_i, \mathbf{w}_i - \mathbf{w}^* \rangle \;+\; \alpha_i^2 \,\|\mathbf{d}_i\|_2^2 . \tag{C.1}$$

Step 2 — Corollary 3.2 of [7]. Hypothesis (2.13) is verified in step (a) of the proof of Theorem 3.1, so Corollary 3.2 of [7] (specialised at $\bar{x} = \mathbf{w}^*$, $\sigma_k = 0$, $\bar{\sigma}_k = 0$) gives

$$\langle \mathbf{d}_i,\, \mathbf{w}^* - \mathbf{w}_i \rangle \;\le\; \gamma_i \big(f^* - f(\mathbf{w}_i)\big) \;+\; \big[f(\mathbf{w}^*) - f^* + 0\big] \;=\; -\gamma_i \big(f(\mathbf{w}_i) - f^*\big),$$

that is,

$$\langle \mathbf{d}_i,\, \mathbf{w}_i - \mathbf{w}^* \rangle \;\ge\; \gamma_i \,\big(f(\mathbf{w}_i) - f^*\big). \tag{C.2}$$

The factor γ_i on the right-hand side (and not the full 1 that convexity would give for the plain subgradient $\mathbf{g}_i$) is the deflection-induced loss: $\mathbf{d}_i$ is a convex combination involving past subgradients (those at $\mathbf{w}_{i-1}, \mathbf{w}_{i-2}, \dots$), so only the weight γ_i on the current $\mathbf{g}_i$ can be certified to separate $\mathbf{w}_i$ from $\mathbf{w}^*$ by the subgradient inequality.

Step 3 — Polyak stepsize substitution. We work in the regime where Lemma 3.8 of [7] has driven the target $f_{\mathrm{ref}}^k - \delta_k$ to f^*, so (3.2) reads $\alpha_i = \beta_i (f(\mathbf{w}_i) - f^*) / \|\mathbf{d}_i\|_2^2$. Substituting into the two non-constant terms of (C.1) and applying (C.2):

$$-2\alpha_i \langle \mathbf{d}_i, \mathbf{w}_i - \mathbf{w}^* \rangle \overset{(C.2)}{\leq} -2\alpha_i \gamma_i \big(f(\mathbf{w}_i) - f^*\big) = -2\,\beta_i\,\gamma_i \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2}, \tag{C.3}$$

$$\alpha_i^2 \|\mathbf{d}_i\|_2^2 = \left(\frac{\beta_i (f(\mathbf{w}_i) - f^*)}{\|\mathbf{d}_i\|_2^2}\right)^2 \|\mathbf{d}_i\|_2^2 = \beta_i^2 \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2}. \tag{C.4}$$

Step 4 — Collecting. Summing (C.3) and (C.4) into (C.1),

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - \big(2\,\beta_i\,\gamma_i - \beta_i^2\big) \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2},$$

and factoring β_i,

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - \beta_i\,(2\gamma_i - \beta_i) \frac{(f(\mathbf{w}_i) - f^*)^2}{\|\mathbf{d}_i\|_2^2},$$

which is (3.8). This matches eq. (3.17) of [7] under the notation map above.

Specialisation to Algorithm 2. Algorithm 2 sets $\beta_i = \min(1, \gamma_i) = \gamma_i$ (because the clip (3.4) keeps $\gamma_i \leq 1$ at every iteration), so the descent factor collapses to

$$\beta_i\,(2\gamma_i - \beta_i) = \gamma_i\,(2\gamma_i - \gamma_i) = \gamma_i^2 \geq \gamma_{\min}^2,$$

which is the form fed back into the telescoping argument of the proof of Theorem 3.1.

Appendix D

Derivation of the optimal deflection γ^*

We derive the closed form (3.4) for the minimiser of

$$h(\gamma) \;:=\; \|\gamma\,\mathbf{g}_i + (1-\gamma)\,\mathbf{d}_{i-1}\|_2^2$$

on $\mathbb{R}$, then justify the clip to $[\gamma_{\min}, 1]$. Expanding by bilinearity of the inner product,

$$h(\gamma) \;=\; \gamma^2\,\|\mathbf{g}_i\|_2^2 \;+\; 2\,\gamma(1-\gamma)\,\langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle \;+\; (1-\gamma)^2\,\|\mathbf{d}_{i-1}\|_2^2 .$$

Collecting powers of γ,

$$h(\gamma) \;=\; \underbrace{\big(\|\mathbf{g}_i\|_2^2 - 2\,\langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle + \|\mathbf{d}_{i-1}\|_2^2\big)}_{=\,\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2}\,\gamma^2 + 2\,\big(\langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big)\,\gamma + \|\mathbf{d}_{i-1}\|_2^2 , \tag{D.1}$$

a parabola in γ with leading coefficient $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2 \geq 0$.

Degenerate case $\mathbf{g}_i = \mathbf{d}_{i-1}$. The leading coefficient vanishes and $h(\gamma) \equiv \|\mathbf{d}_{i-1}\|_2^2$ is constant on $\mathbb{R}$; moreover the deflected direction $\mathbf{d}_i = \gamma\,\mathbf{g}_i + (1-\gamma)\,\mathbf{d}_{i-1} = \mathbf{d}_{i-1}$ for every γ, so the algorithm is insensitive to the choice. We set $\gamma_i = 1$ by convention (consistent with the unclipped Polyak first step $\mathbf{d}_0 = \mathbf{g}_0$).

Generic case $\mathbf{g}_i \neq \mathbf{d}_{i-1}$. The leading coefficient is strictly positive, so h is strictly convex with a unique unconstrained minimiser. Setting $h'(\gamma) = 0$ in (D.1),

$$2\,\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2\,\gamma + 2\,\big(\langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big) \;=\; 0 \quad\Longrightarrow\quad \gamma^* \;=\; \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i,\, \mathbf{d}_{i-1}\rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2},$$

which is (3.4).

Projection onto $[\gamma_{\min}, 1]$. Since the parabola opens upward, h is decreasing on $(-\infty, \gamma^*]$ and increasing on $[\gamma^*, +\infty)$. The constrained minimum over $[\gamma_{\min}, 1]$ is therefore

$$\gamma_i \;=\; \begin{cases} \gamma_{\min} & \text{if } \gamma^* < \gamma_{\min}, \\ \gamma^* & \text{if } \gamma^* \in [\gamma_{\min}, 1], \\ 1 & \text{if } \gamma^* > 1, \end{cases} \qquad \text{i.e. } \gamma_i \;=\; \min\bigl(1,\, \max(\gamma_{\min},\, \gamma^*)\bigr).$$

The two clip endpoints serve different purposes: the lower clip $\gamma_{\min}$ enforces the uniform bound $\beta^* > 0$ of condition (3.5) of [7] (Section 3.2); the upper clip at 1 keeps $\gamma_i \in (0, 1]$ so that $\mathbf{d}_i$ is a convex combination of $\mathbf{g}_i$ and $\mathbf{d}_{i-1}$, a property used in the per-step descent estimate of Appendix C.

Interpretation. The denominator $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2$ measures how far the current subgradient is from the previous deflected direction: large values shrink γ^* and tilt the deflected direction toward memory $\mathbf{d}_{i-1}$. The numerator $\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle$ captures alignment: it is small (so γ^* is small) when $\mathbf{g}_i$ and $\mathbf{d}_{i-1}$ point in similar directions, and large when they oppose each other.

In the zig-zag regime $\mathbf{g}_i \approx -\mathbf{d}_{i-1}$ with $\|\mathbf{g}_i\|_2 \approx \|\mathbf{d}_{i-1}\|_2$, the numerator is approximately $\|\mathbf{d}_{i-1}\|_2^2 + \|\mathbf{d}_{i-1}\|_2^2 = 2\, \|\mathbf{d}_{i-1}\|_2^2$ while the denominator is approximately $\|2\, \mathbf{d}_{i-1}\|_2^2 = 4\, \|\mathbf{d}_{i-1}\|_2^2$, so $\gamma^* \approx 1/2$: the deflected direction $\mathbf{d}_i$ splits evenly between $\mathbf{g}_i$ and $\mathbf{d}_{i-1}$ and the oscillatory component cancels.

Bibliography

[1] Dimitri P. Bertsekas. *Nonlinear Programming*. 2nd. Athena Scientific, 1999.

[2] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[3] Ulf Brännlund. "A Generalized Subgradient Method with Relaxation Step". In: *Mathematical Programming* 71.2 (1995), pp. 207–219.

[4] Sébastien Bubeck. *Convex Optimization: Algorithms and Complexity*. arXiv:1405.4980v2. 2015.

[5] Rick Chartrand and Wotao Yin. "Iteratively reweighted algorithms for compressive sensing". In: *2008 IEEE international conference on acoustics, speech and signal processing*. IEEE. 2008, pp. 3869–3872.

[6] Alice Cortinovis. *Introduction to Least Squares Problem*. Lecture notes: Intro to Least Squares, CM 646AA, Università di Pisa. 2025.

[7] Giacomo d'Antonio and Antonio Frangioni. "Convergence Analysis of Deflected Conditional Approximate Subgradient Methods". In: *SIAM Journal on Optimization* 20.1 (2009). DOI: 10.1137/080718814. URL: https://pages.di.unipi.it/frangio/papers/DeflProjSubG.pdf.

[8] Ingrid Daubechies et al. *Iteratively re-weighted least squares minimization for sparse recovery*. 2008. arXiv: 0807.0575 [math.NA]. URL: https://arxiv.org/abs/0807.0575.

[9] Bradley Efron et al. "Least angle regression". In: *The Annals of Statistics* 32.2 (2004), pp. 407–499. DOI: 10.1214/009053604000000067.

[10] Antonio Frangioni. *Nonsmooth Unconstrained Optimization*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. 2026.

[11] Antonio Frangioni. *Smooth Unconstrained Optimization*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90521/mod_resource/content/8/4-smooth%20unconstrained%20optimization.pdf. 2025.

[12] Antonio Frangioni. *Unconstrained Multivariate Optimality and Convexity*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90520/mod_resource/content/8/3-unconstrained%20optimality.pdf. 2025.

[13] Jean-Louis Goffin and Krzysztof C. Kiwiel. "Convergence of a Simple Subgradient Level Method". In: *Mathematical Programming* 85.1 (1999), pp. 207–211.

[14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer, 2009. URL: https://hastie.su.domains/ElemStatLearn/.

[15] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. "Extreme learning machine: Theory and applications". In: *Neurocomputing* 70.1 (2006), pp. 489–501. DOI: 10.1016/j.neucom.2005.12.126.

[16] Hien D. Nguyen. *An Introduction to MM Algorithms for Machine Learning and Statistical*. 2016. arXiv: 1611.03969 [stat.CO]. URL: https://arxiv.org/abs/1611.03969.

[17] R. Kelley Pace and Ronald Barry. "Sparse spatial autoregressions". In: *Statistics & Probability Letters* 33.3 (1997), pp. 291–297. DOI: 10.1016/S0167-7152(96)00140-X.

[18] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997.